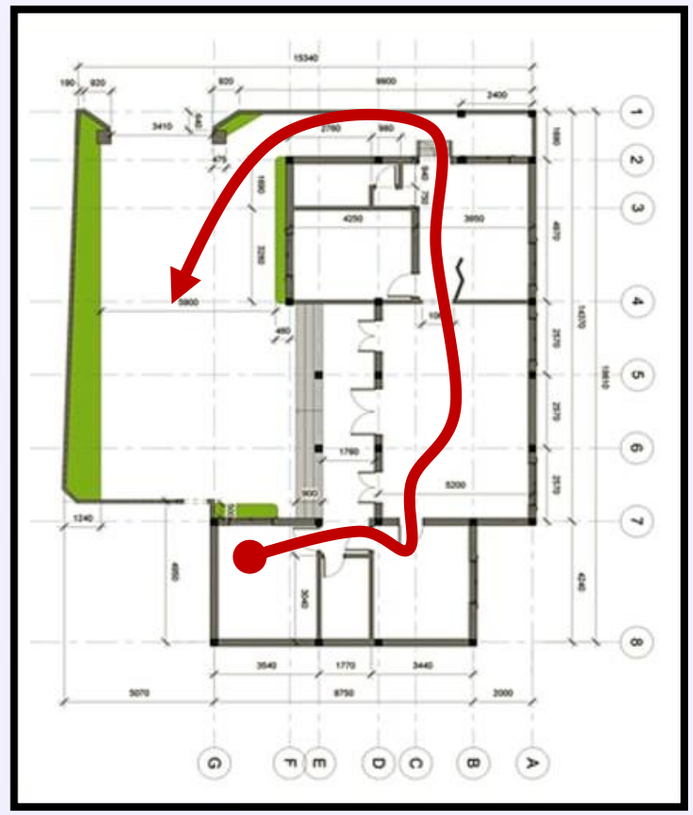
Path Planning on a Graph

Dr. Troi Williams

Slide credits: Pratap Tokekar, Chahat Deep Singh

Find a path from point A to B



assuming we know the map

This lecture (and next)

(Five) Assumptions

This lecture (and next)

(Five) Assumptions

- Known map

This lecture (and next)

(Five) Assumptions

- Known map
- No moving obstacles

This lecture (and next)

(Five) Assumptions

- Known map
- No moving obstacles
- Point robot

This lecture (and next)

(Five) Assumptions

- Known map
- No moving obstacles
- Point robot
- Perfect sensing (we know where we are at all times)

This lecture (and next)

(Five) Assumptions

- Known map
- No moving obstacles
- Point robot
- Perfect sensing (we know where we are at all times)
- Discrete time, discrete state, discrete actions

Three properties of a good planner

1. *Completeness:* Planner is complete if it always finds a path from start to goal in finite time, if one exists. Else, it declares failure in finite time.

Three properties of a good planner

1. *Completeness:* Planner is complete if it always finds a path from start to goal in finite time, if one exists. Else, it declares failure in finite time.

   But what does "finite time" mean?

Three properties of a good planner

1. *Completeness:* Planner is complete if it always finds a path from start to goal in finite time, if one exists. Else, it declares failure in finite time.

2. *Optimality:* Planner is optimal if it always finds a minimum cost path from start to goal in finite time, if one exists.

Properties of a good planner

1. *Completeness:* Planner is complete if it always finds a path from start to goal in finite time, if one exists. Else, it declares failure in finite time.

2. *Optimality:* Planner is optimal if it always finds a minimum cost path from start to goal in finite time, if one exists.

3. *Efficiency:* An algorithm is efficient if it finds the path in the least possible computational time (for all inputs).

Our plan

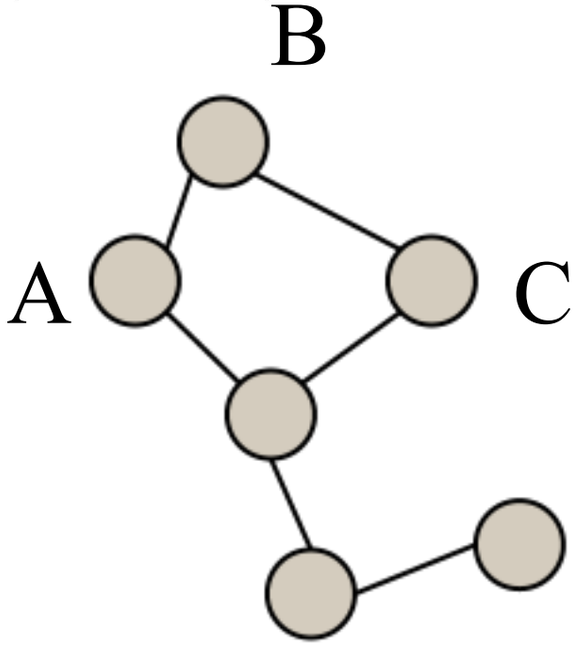
- Today: *complete* but not *optimal* planners
  - Breadth First Search (BFS)
  - Depth First Search (DFS)

- Next lecture: *optimal* and *efficient* planners
  - Dijkstra
  - A*

- Later: *complete* but better running time than BFS/DFS for more complex settings
  - RRT

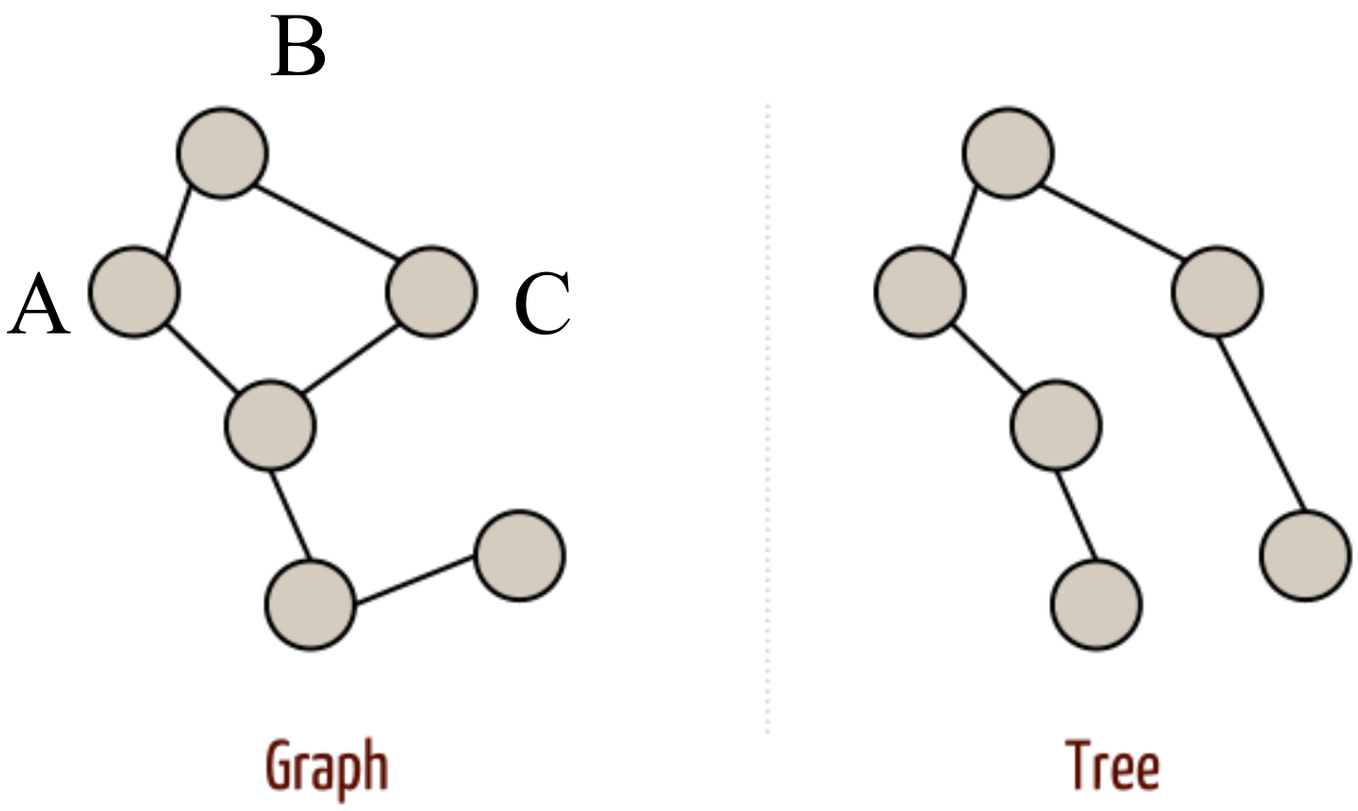
Graphs

- Mathematical structures to model pairwise relations between objects
- Graph consists of
  - Vertices (aka nodes)
  - Edges (aka links)



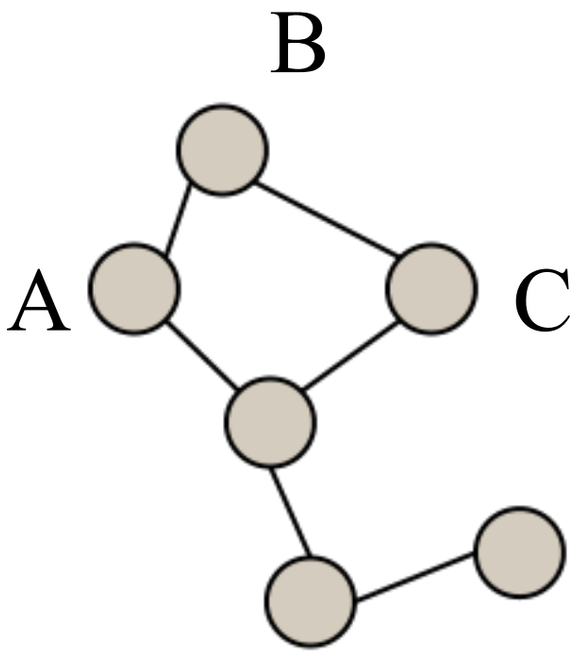
Graphs versus Trees

- What are trees in the context of graphs?

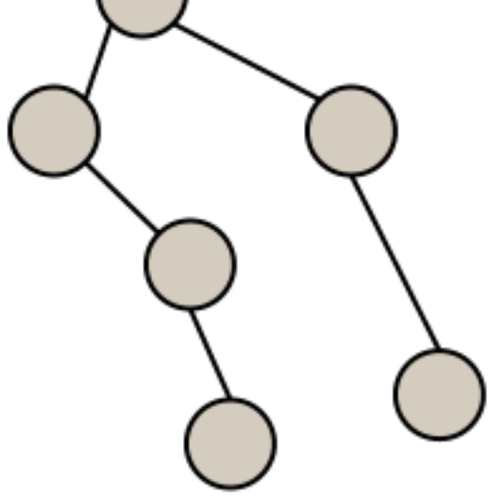


Graphs versus Trees

- Trees are graphs without any *cycle*
- All trees are graphs but not all graphs are trees



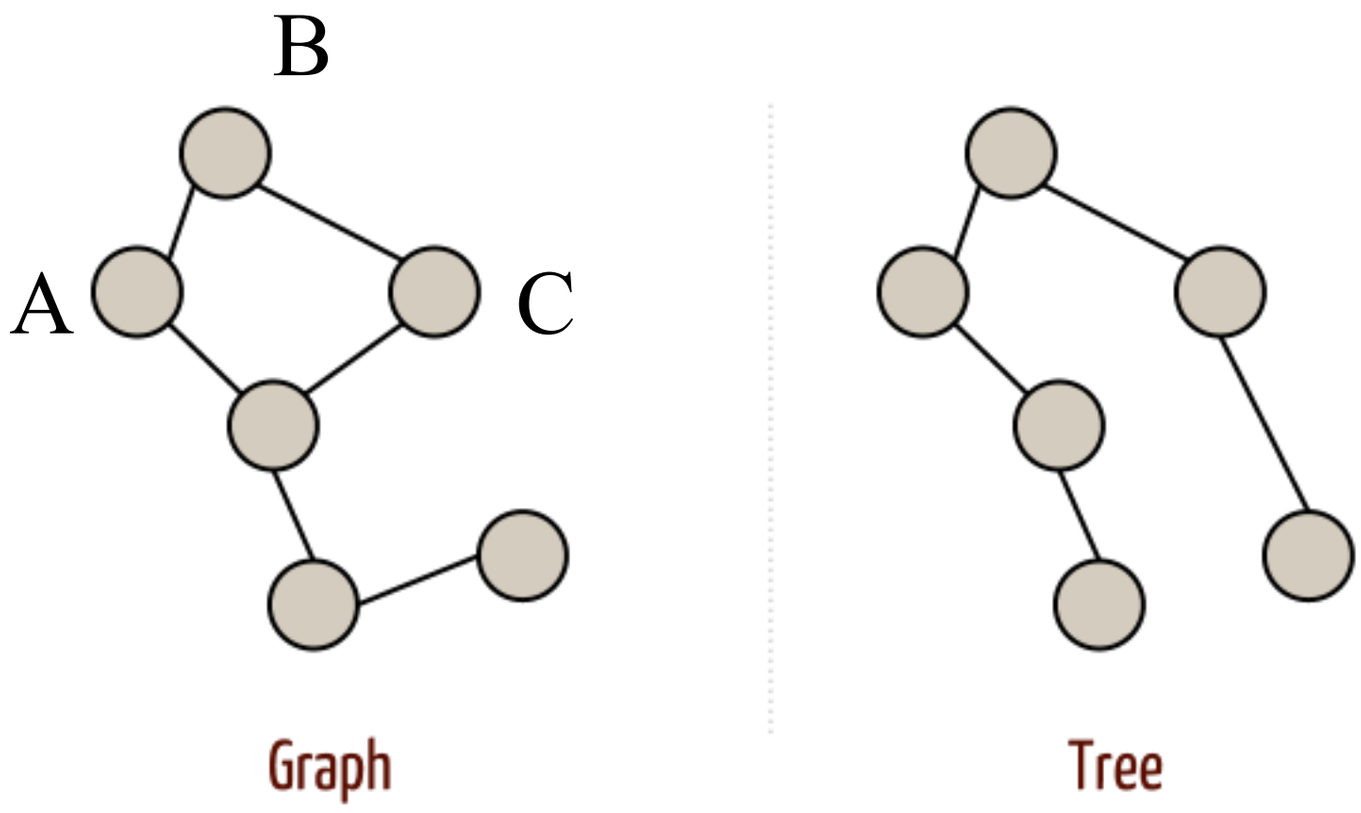
Graph



Tree

Graphs versus Trees

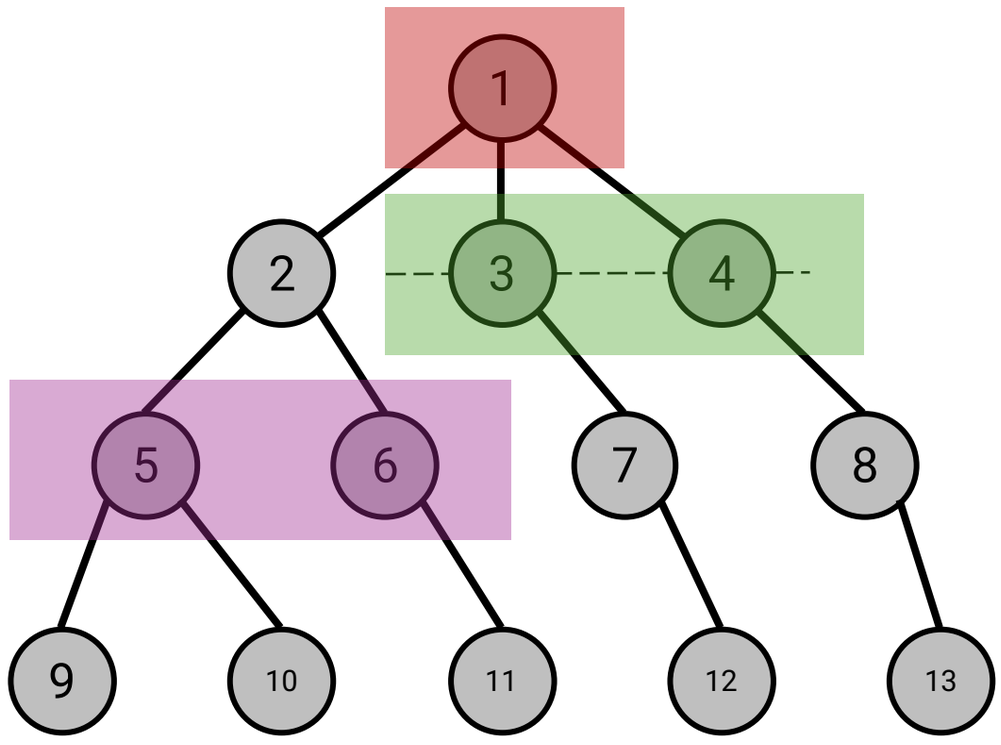
- Trees are graphs without any *cycle*
- All trees are graphs, but not all graphs are trees



- *We'll make the connection between graphs and trees more explicit later.*

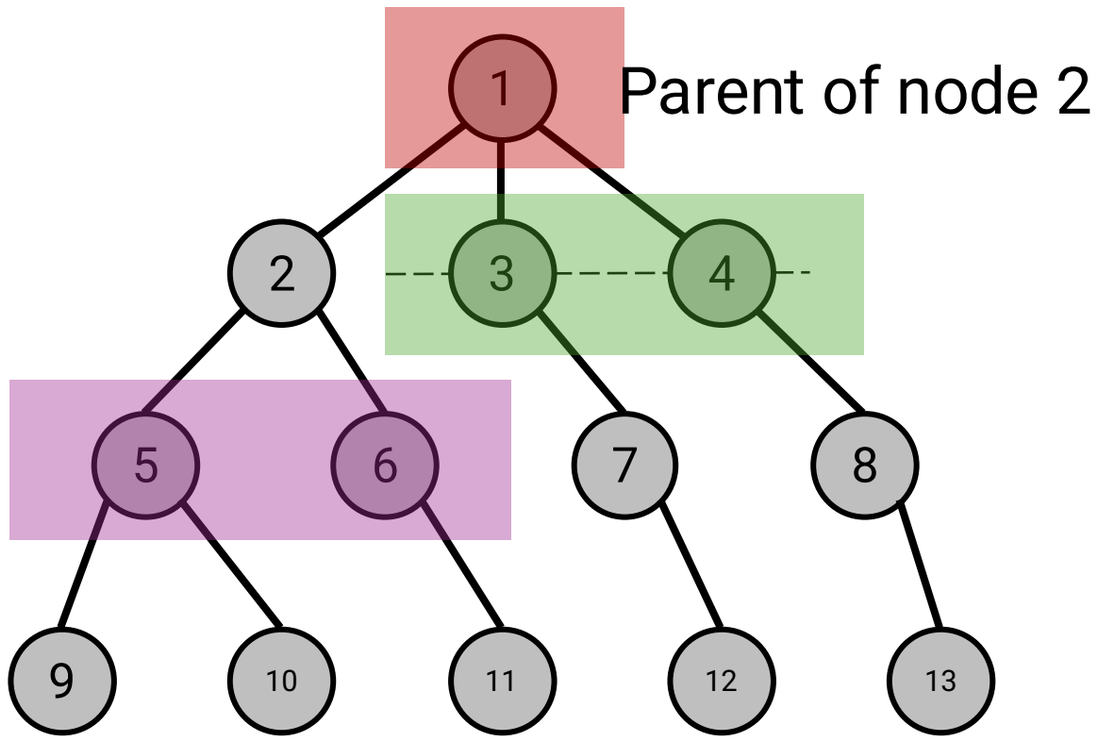
Trees

Consider node 2



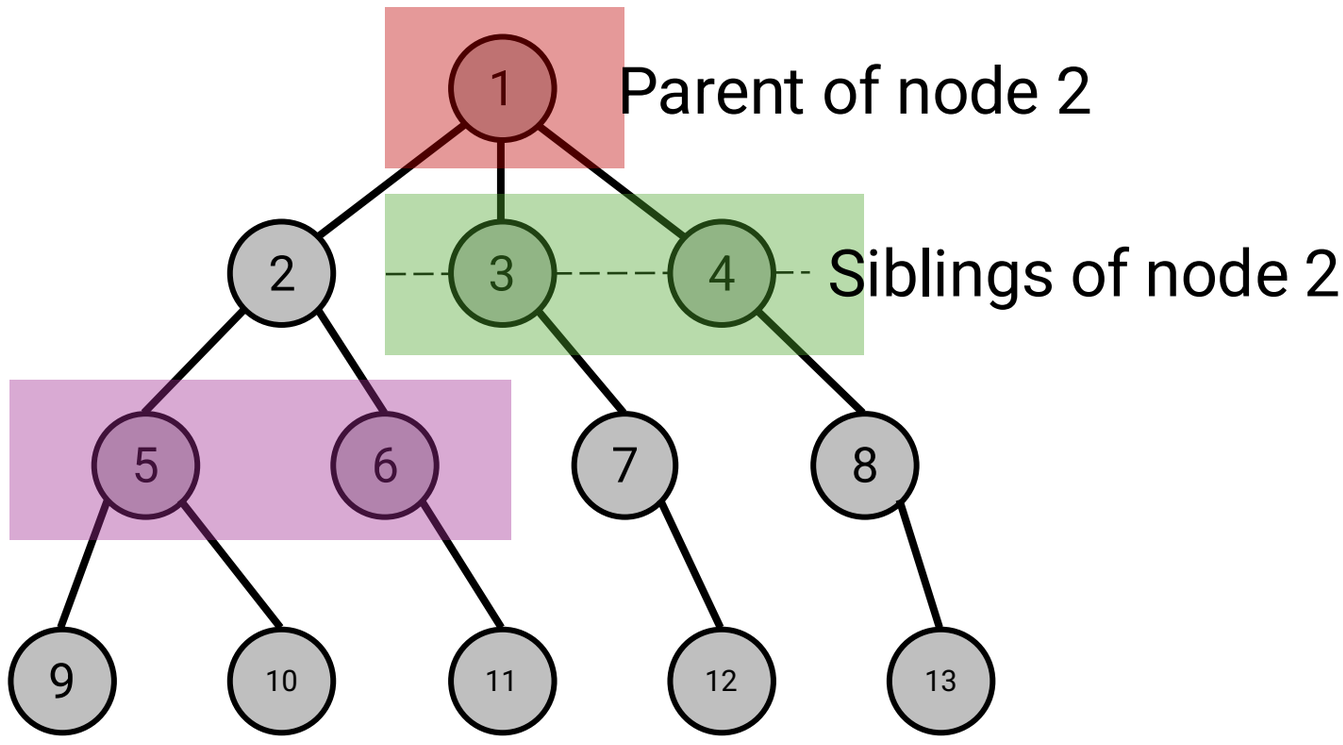
Trees

Consider node 2



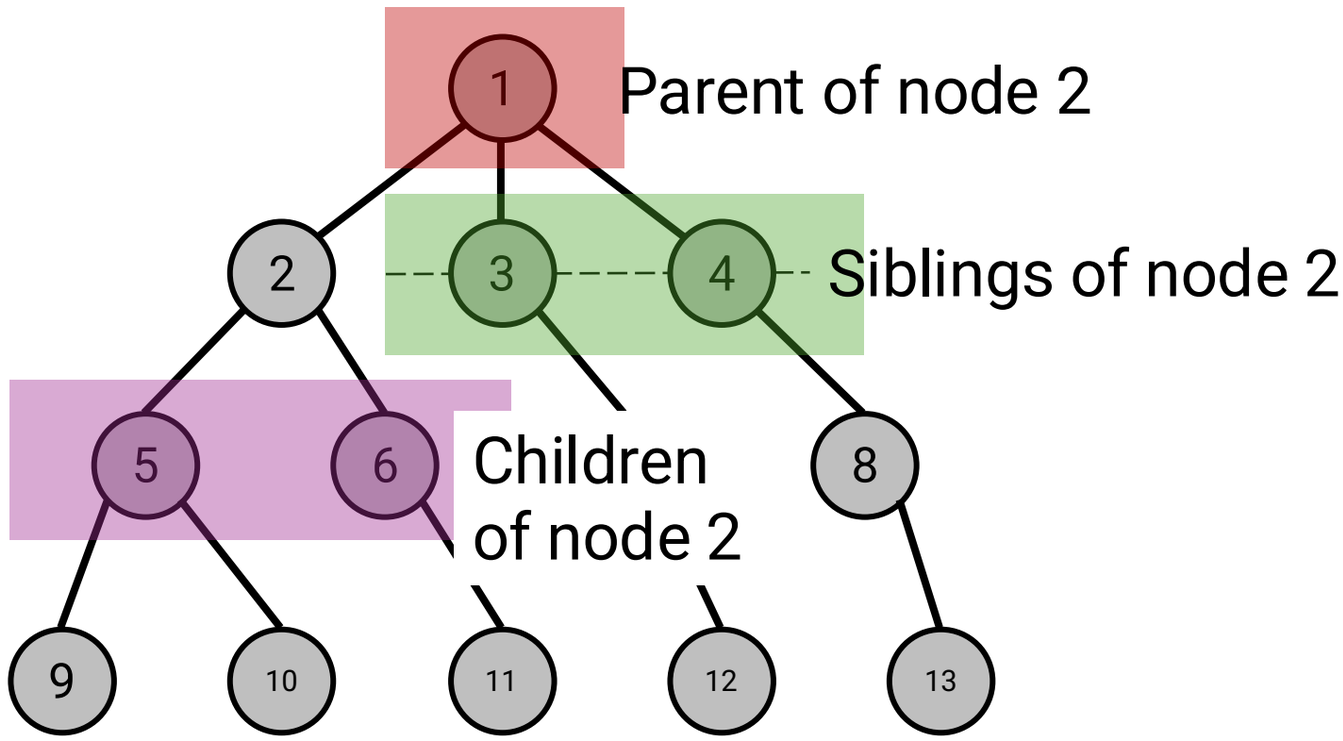
Trees

Consider node 2



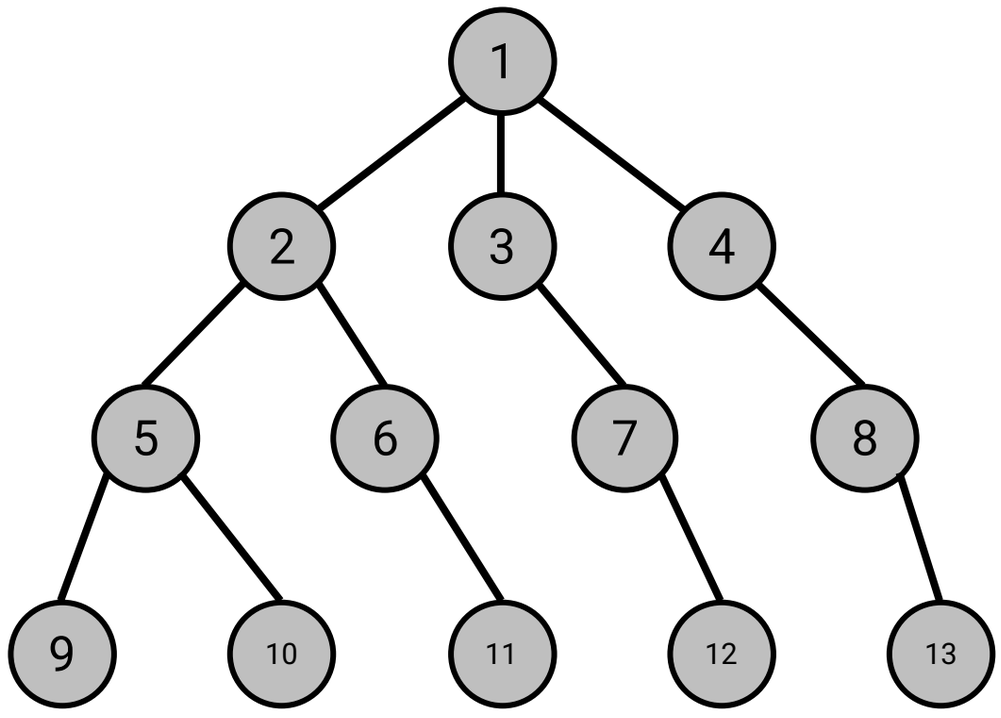
Trees

Consider node 2



Weighted Graphs*

aka *costs*



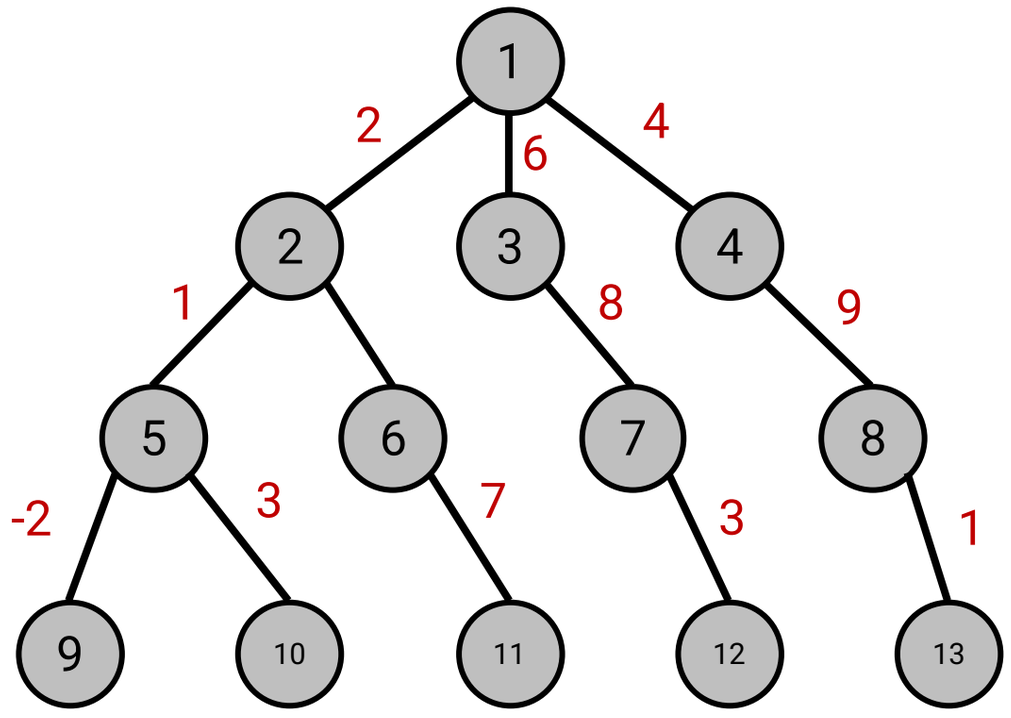
Where are the weights?

What do they represent?

*Shown for a tree but also applies to general graphs

Weighted Graphs*

aka *costs*



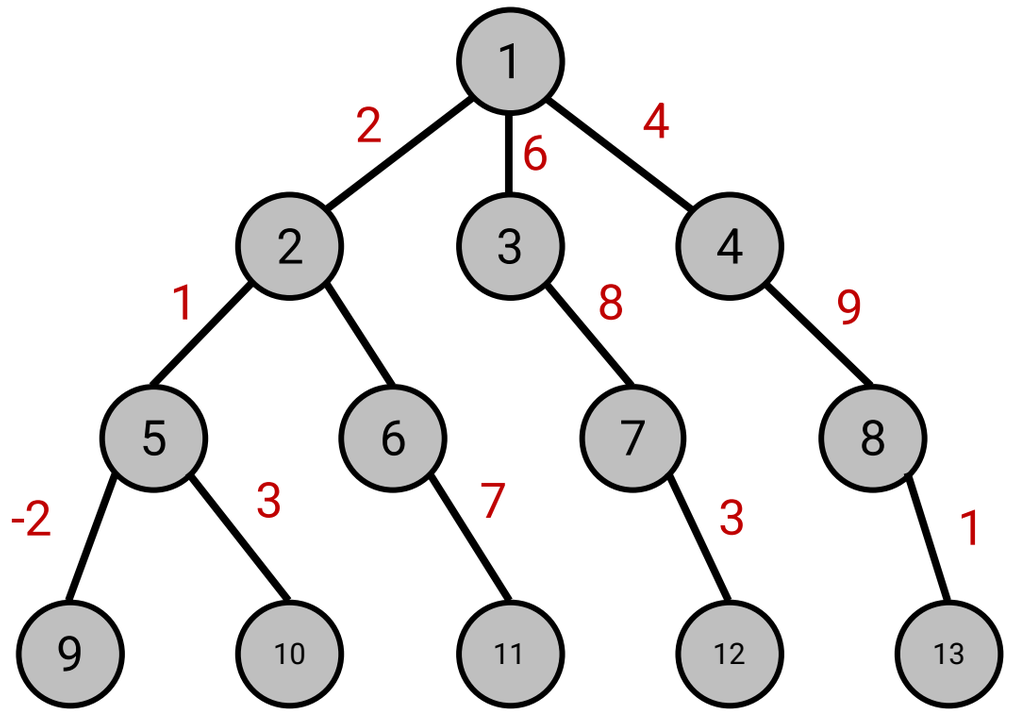
Where are the weights?
On edges!

What do they represent?

*Shown for a tree but also applies to general graphs

Weighted Graphs*

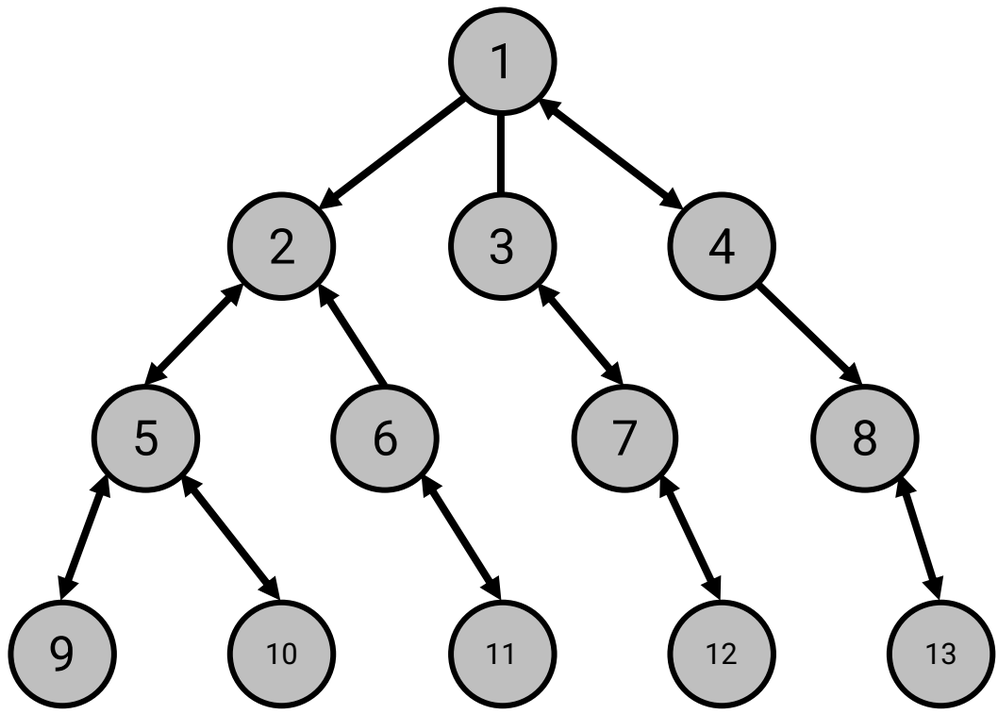
aka *costs*



Weights can represent travel *time*, *energy*, *distance*, etc.

*Shown for a tree but also applies to general graphs

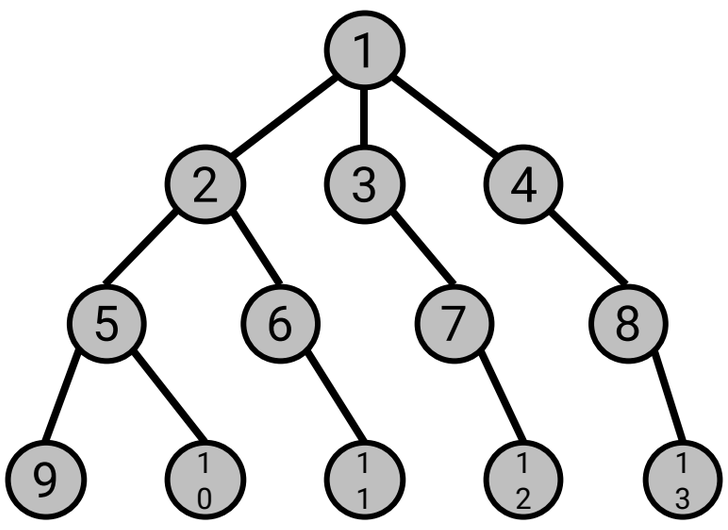
Directed Graphs* (or digraph)



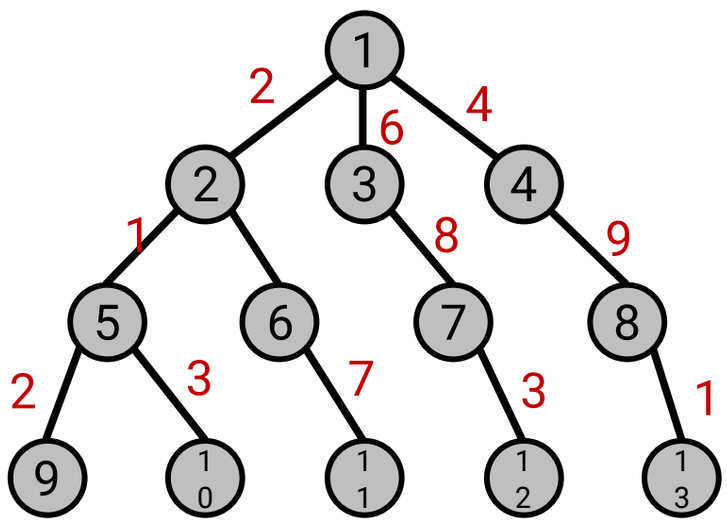
*Shown for a tree but also applies to general graphs

Types of Graphs

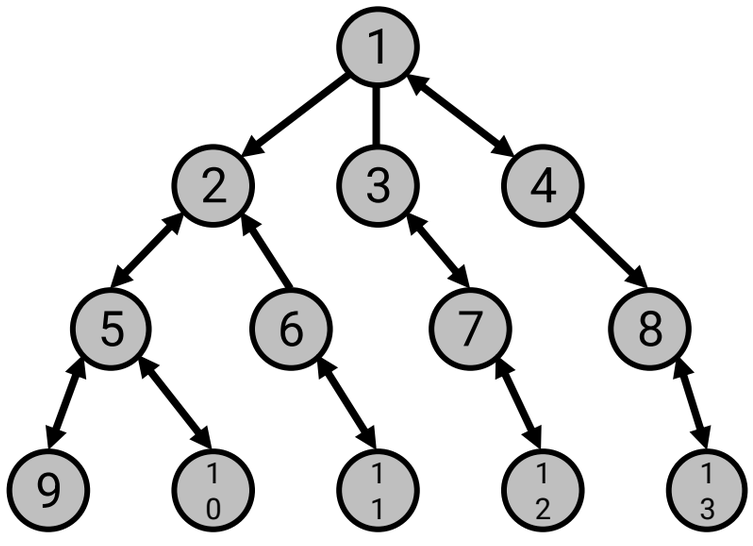
Undirected



Weighted



Directed



Representing Grids

Representing Grids

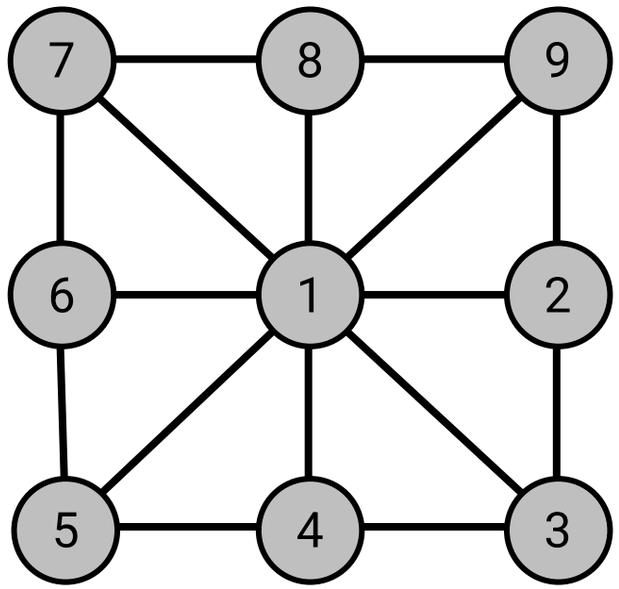
7	8	9
6	1	2
5	4	3

Representing Grids

- Typically, we will model the environment/map as a uniform grid

7	8	9
6	1	2
5	4	3

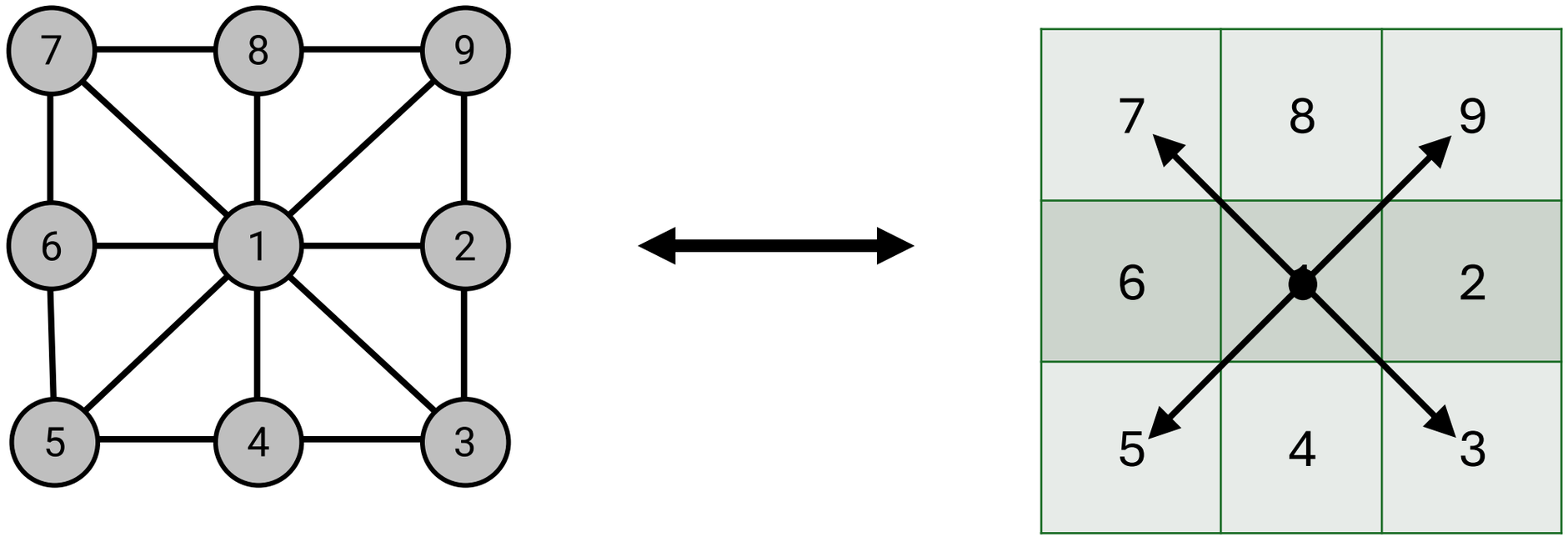
Representing Grids as a Graph



7	8	9
6	1	2
5	4	3

Representing Grids as a Graph

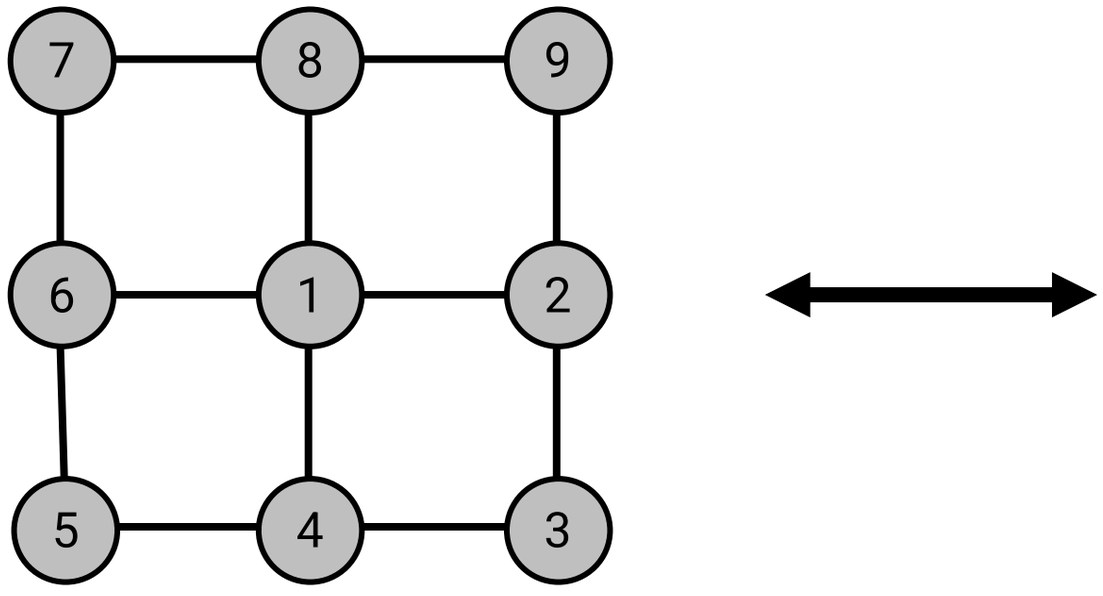
called as an 8-connected grid graph



when the robot can traverse diagonally

Representing Grids as a Graph

called as a 4-connected grid graph

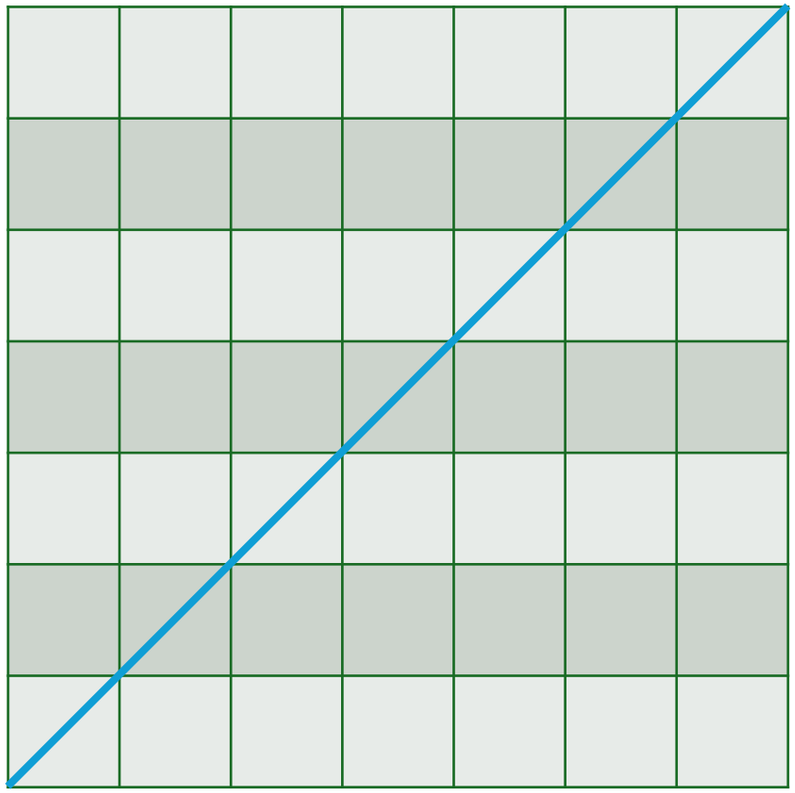


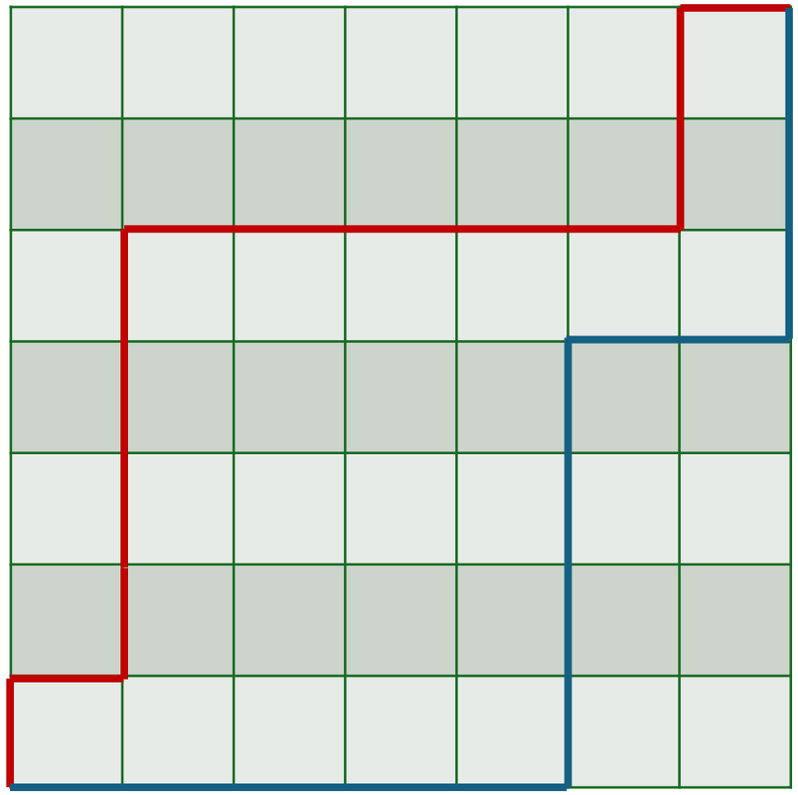
7	8	9
6	1	2
5	4	3

when the robot can only traverse in straight lines

but may also need to keep track of robot heading

Euclidean vs Manhattan Distance





With a 4-connected grid graph, you will likely minimize Manhattan distance. *But why???*

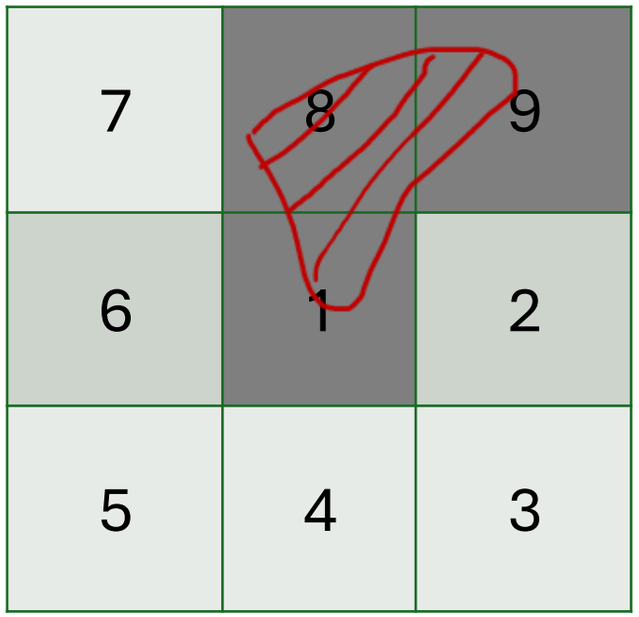
Euclidean

Manhattan

Obstacles

- Typically, we will model the environment/map as a uniform grid

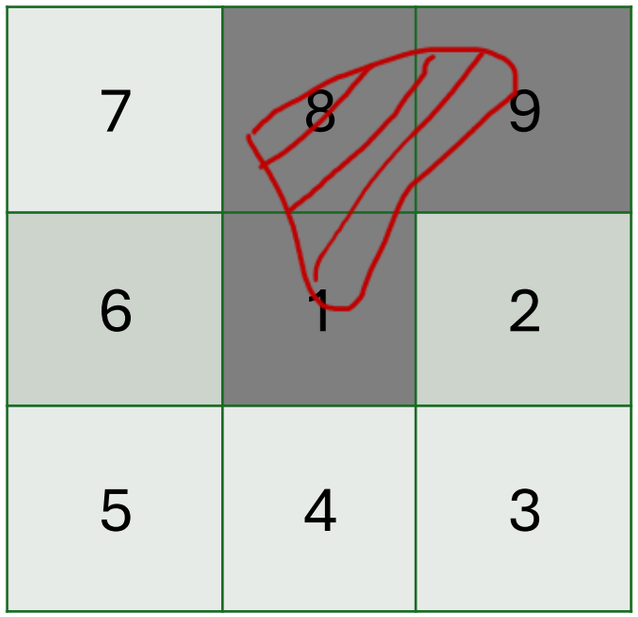
- *How could we deal with obstacles? (Hint →)*



Obstacles

- Typically, we will model the environment/map as a uniform grid

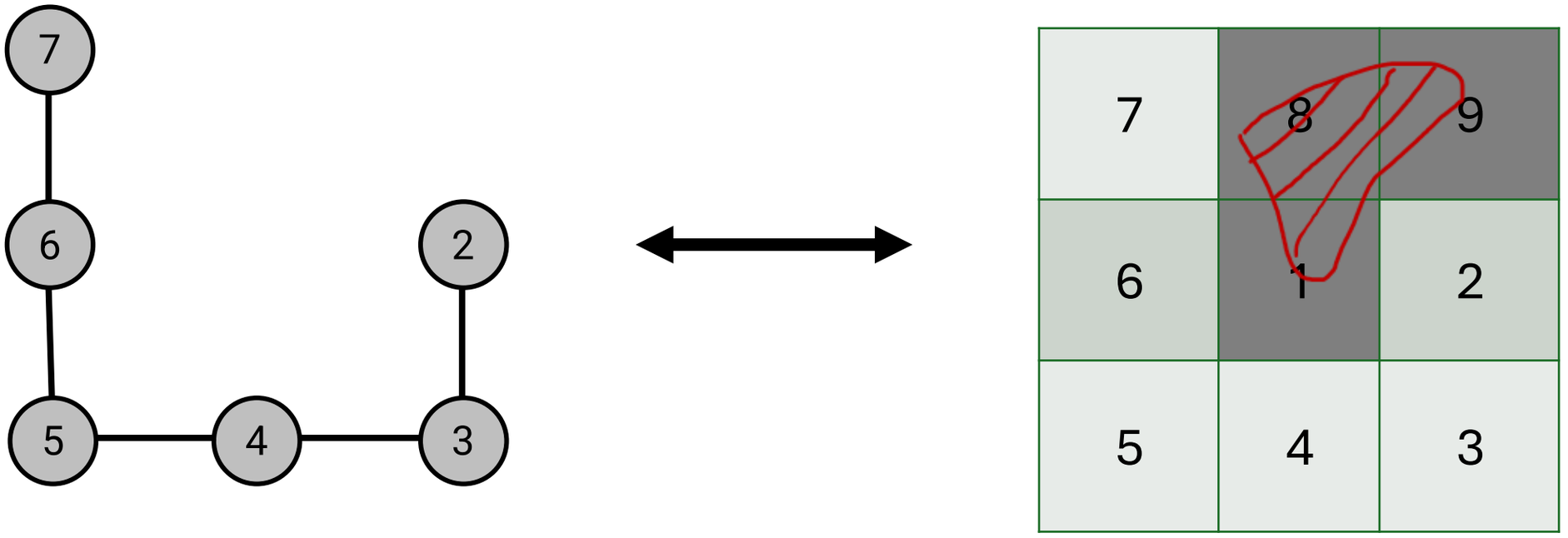
- We will *snap* all the obstacles to the grid
  - If part of an obstacle lies in a grid cell, the entire cell will be treated as an obstacle



- *Now, how do we deal with obstacles in a graph?*
  (Hint: 3 ways)

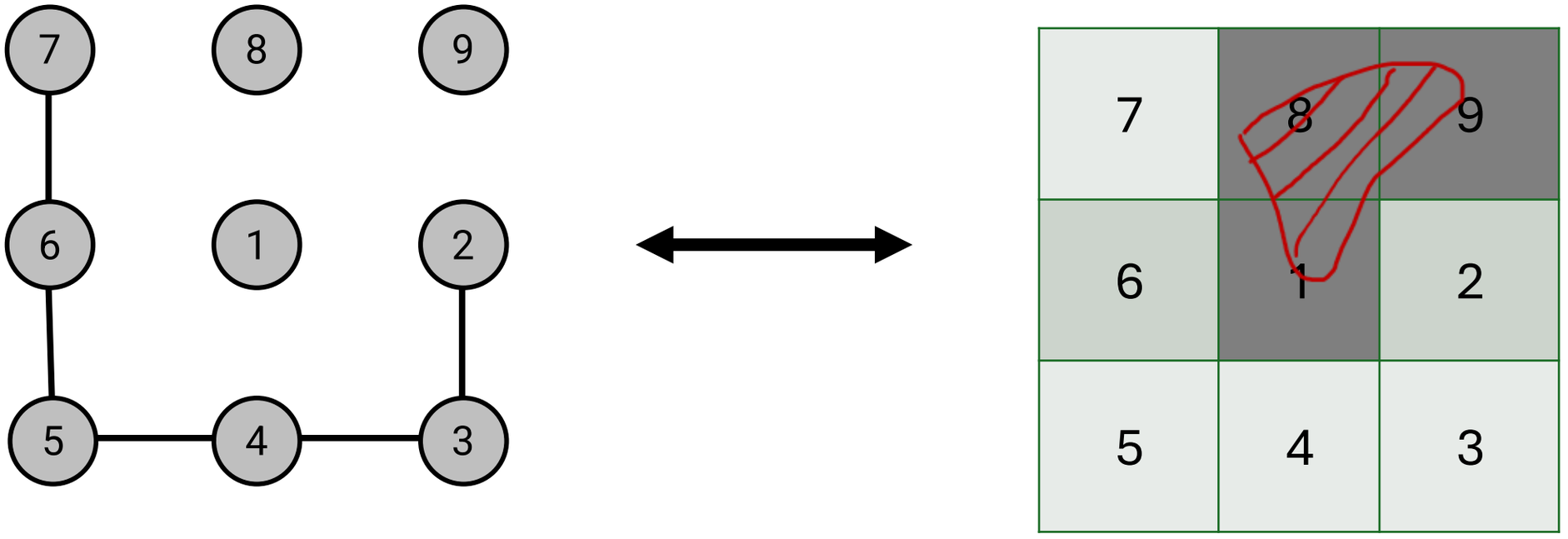
Dealing with obstacles

1. Remove the vertices that belong to the obstacles



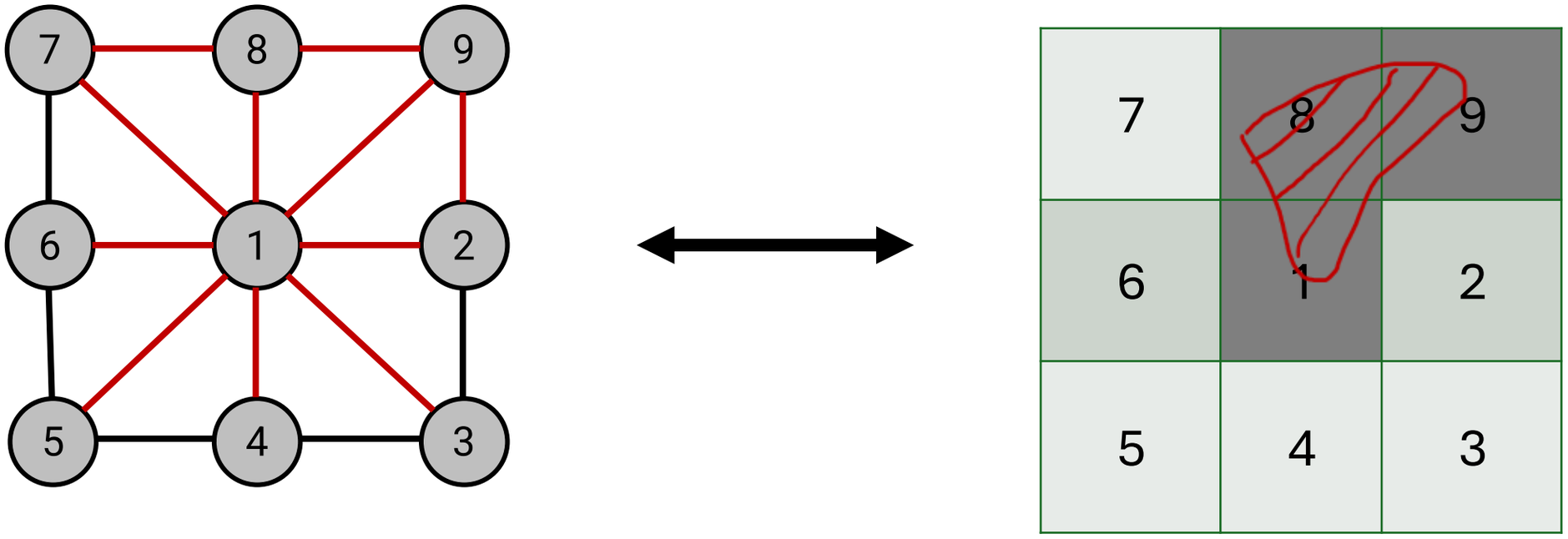
Dealing with obstacles

2. Remove the edges incident with obstacle vertices

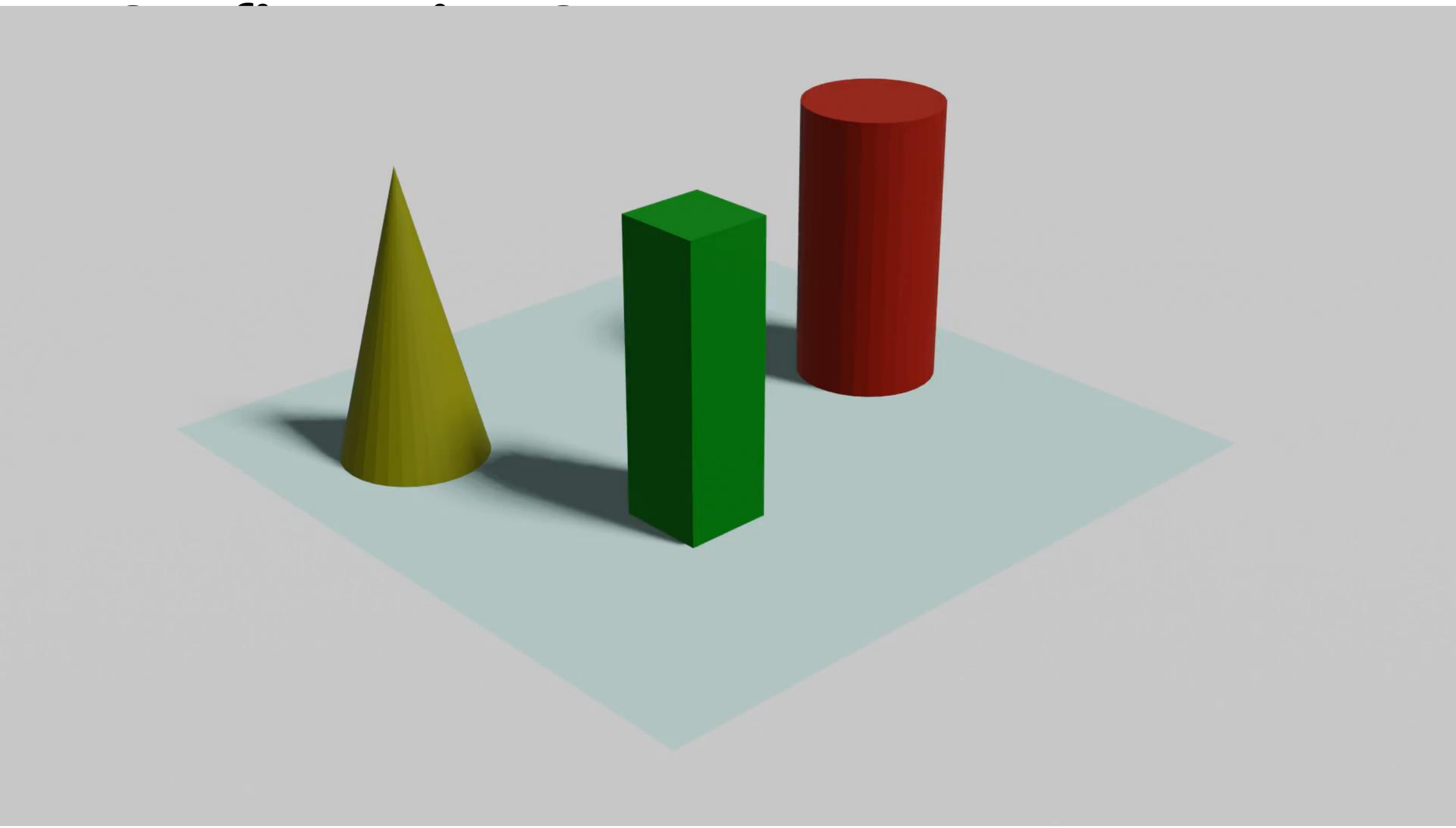


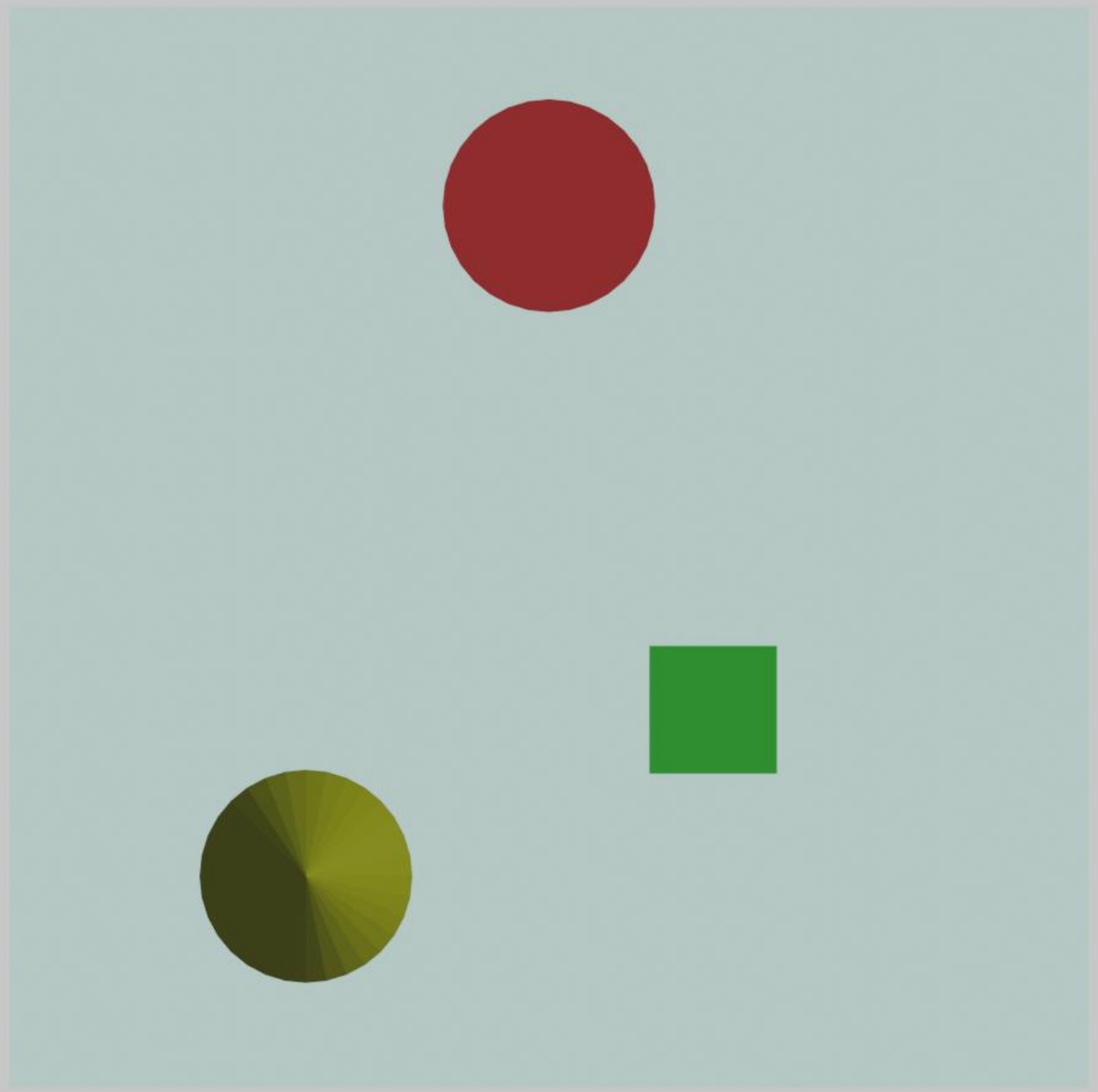
Dealing with obstacles

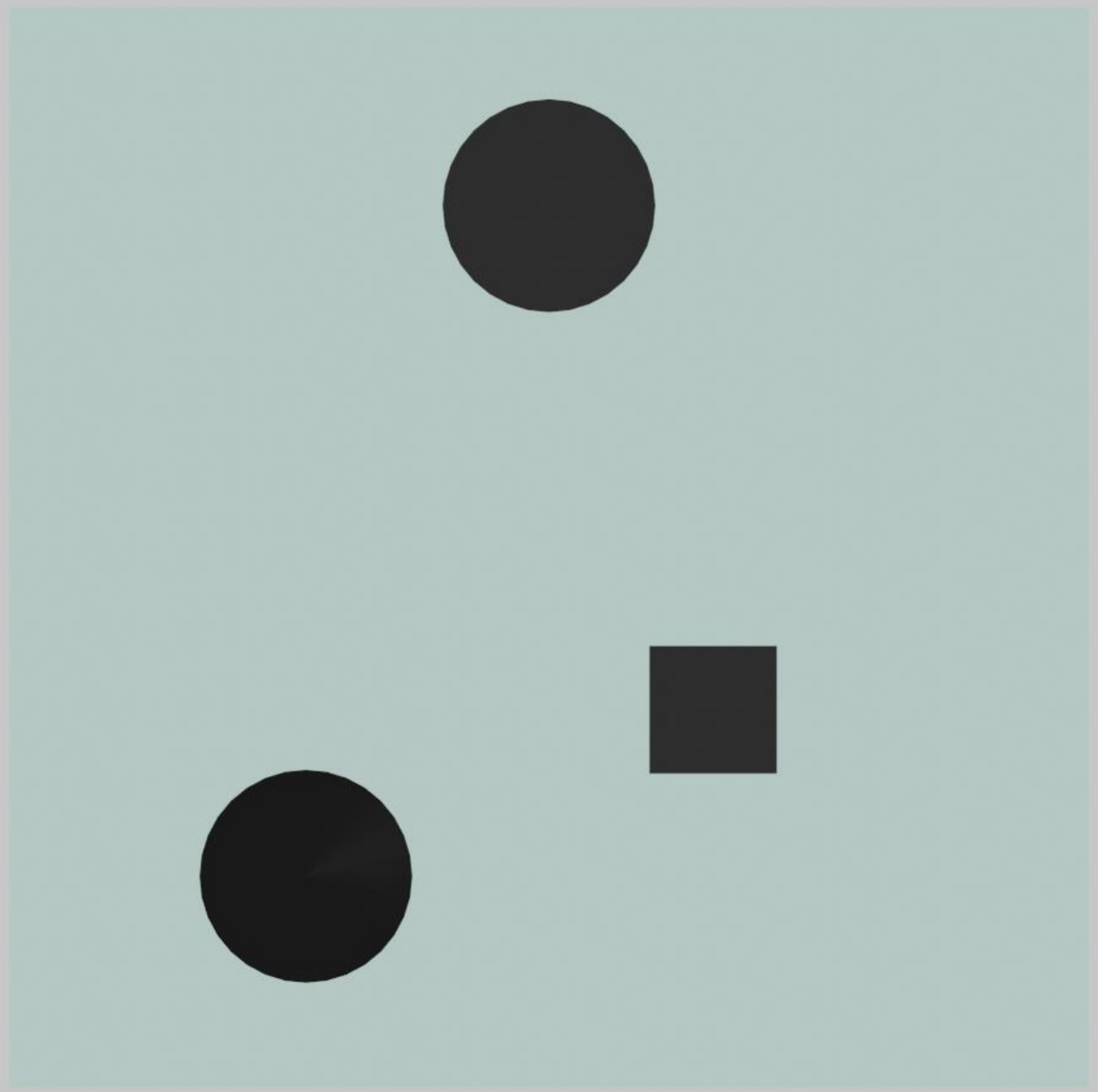
3. Or keep the vertices and edges but make the edges to have infinite (or practically, very large) cost



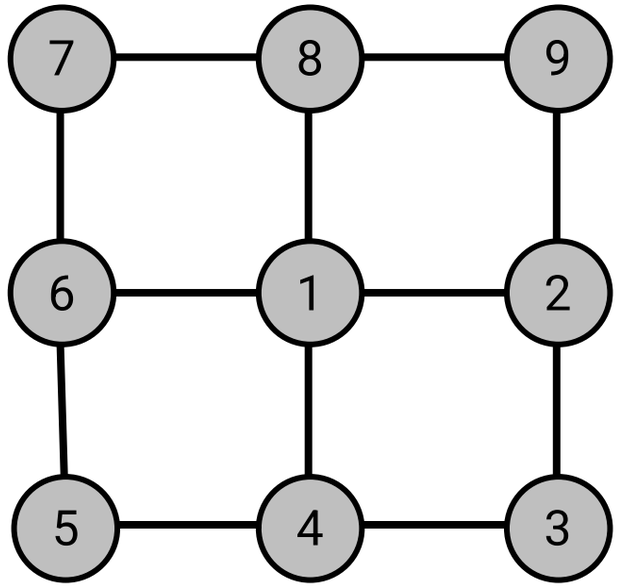
3D World

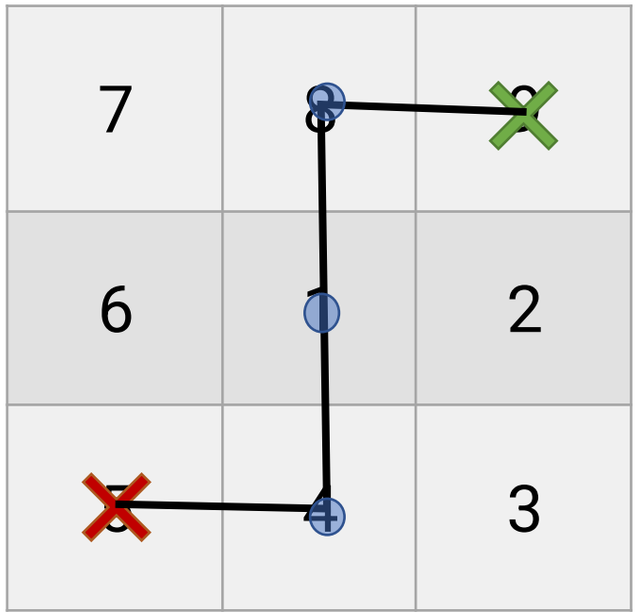






Shortest path finding





Our plan

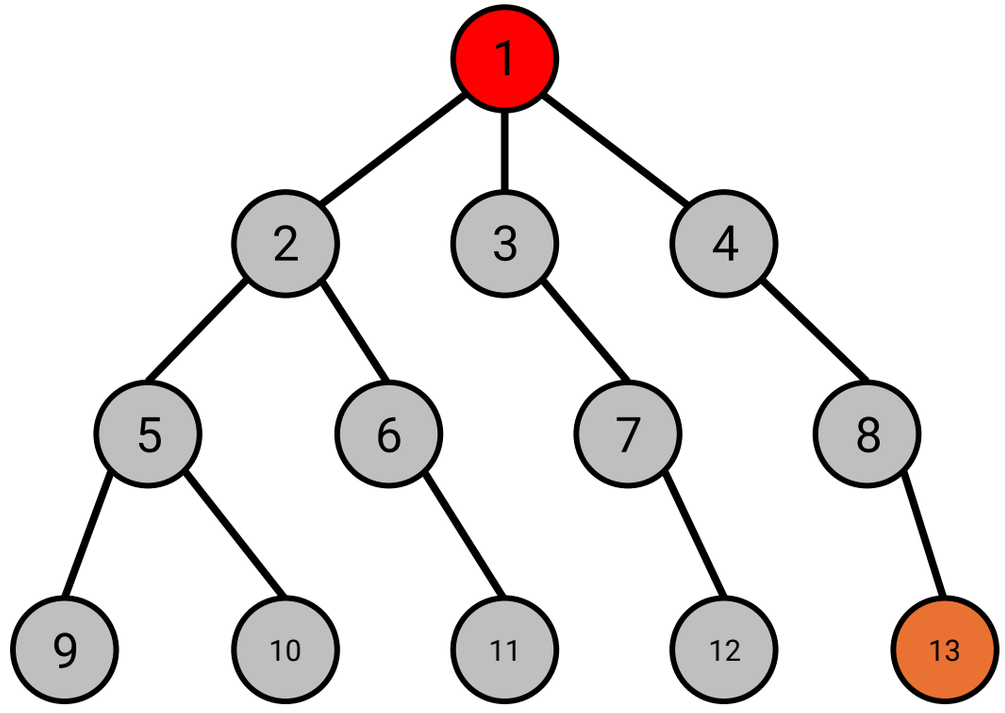
- Today: *complete* but not *optimal* planners
  - Breadth First Search (BFS)
  - Depth First Search (DFS)

- Next lecture: *optimal* and *efficient* planners
  - Dijkstra
  - A*

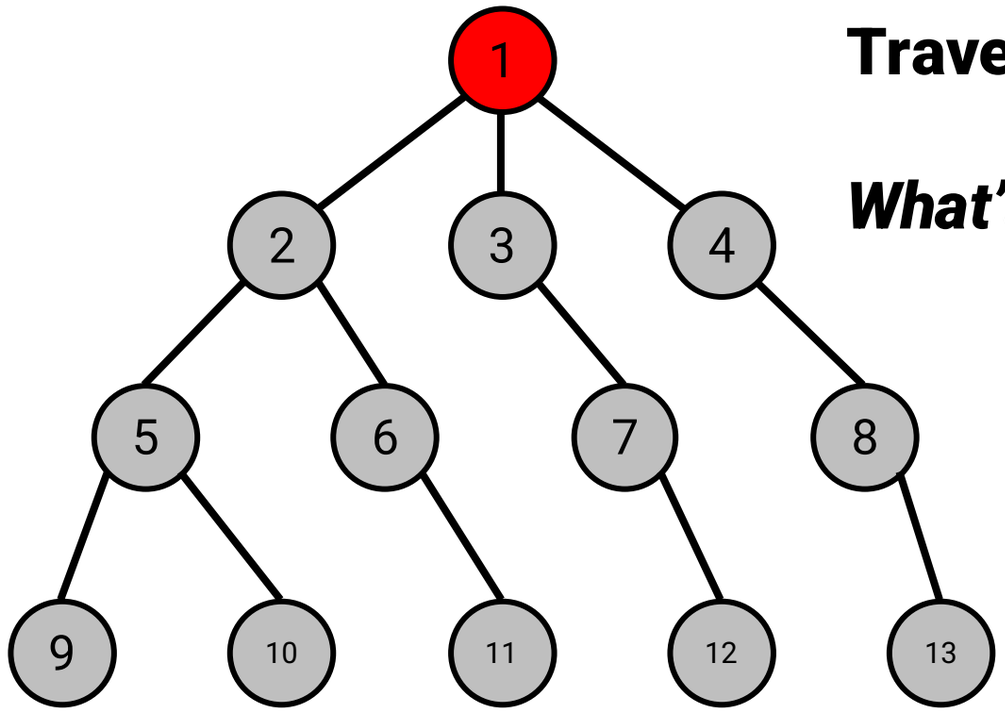
- Later: *complete* but better running time than BFS/DFS for more complex settings
  - RRT

Let’s start simple

- Let’s say that the graph is a tree—bringing the concepts together
- The *root* node is always the start
- The goal node is one of the nodes in the tree



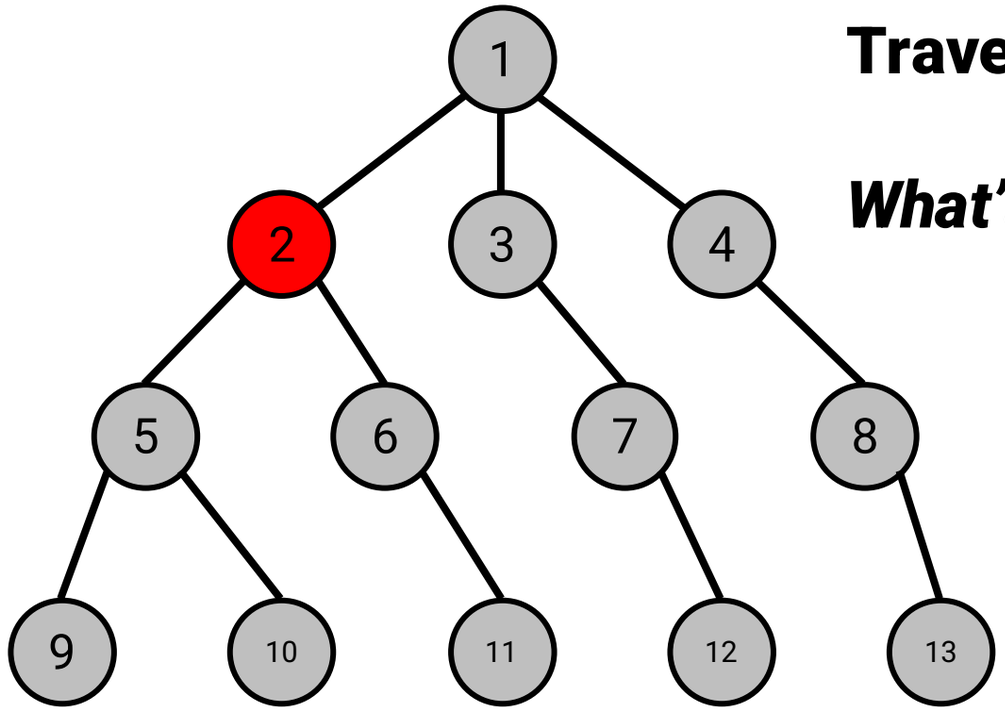
Breadth First Search



Traverse through Siblings First!

What's the next node we explore?

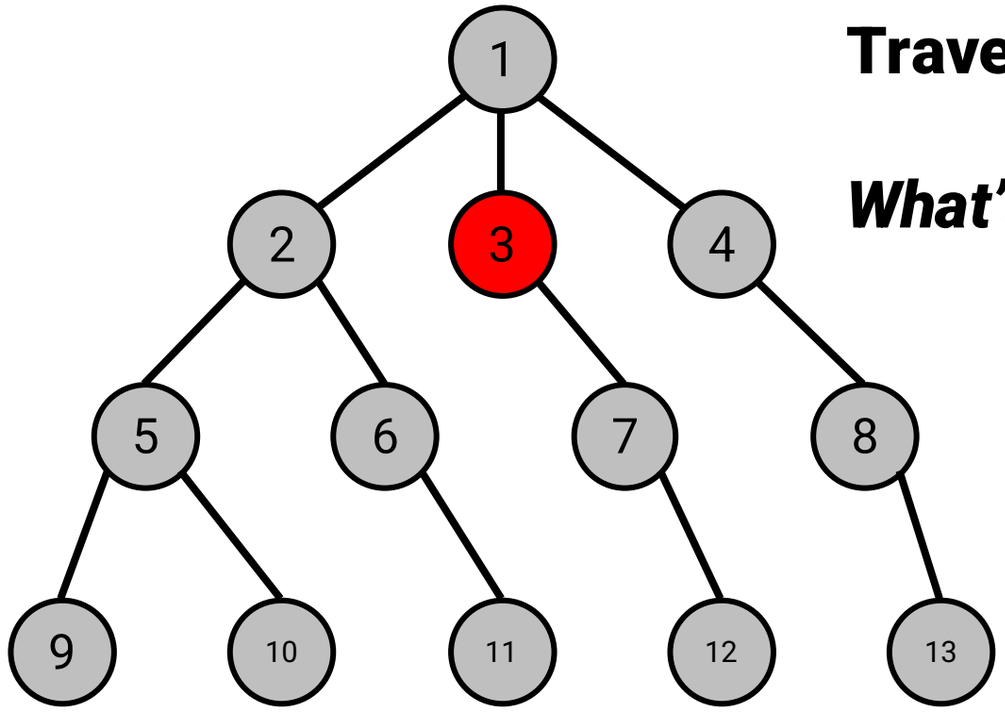
Breadth First Search



Traverse through Siblings First!

What's the next node we explore?

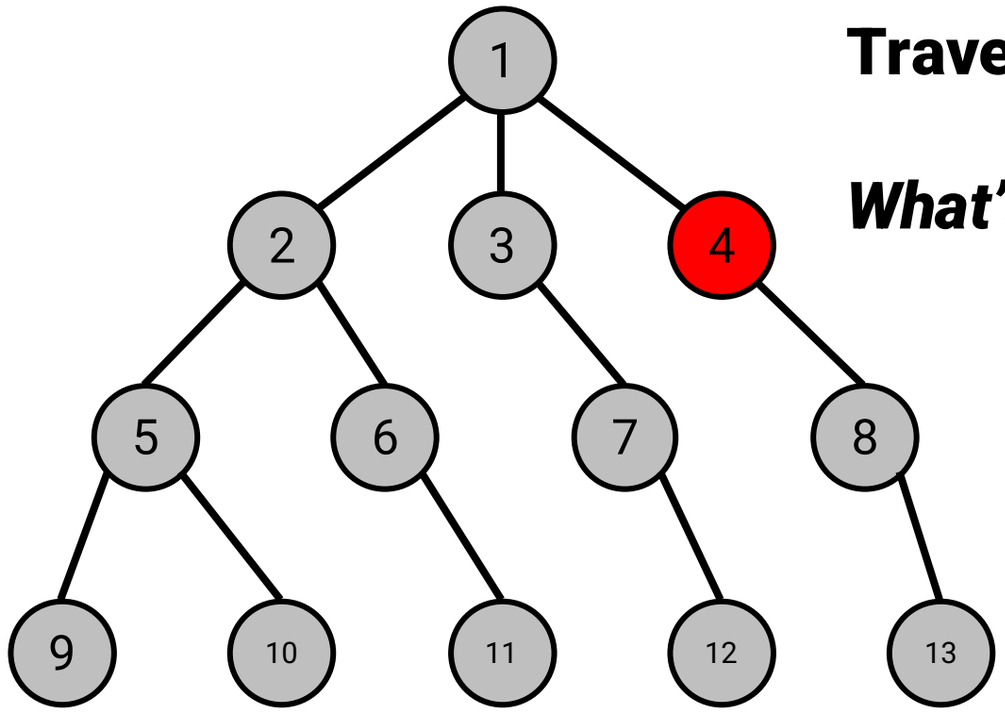
Breadth First Search



Traverse through Siblings First!

What's the next node we explore?

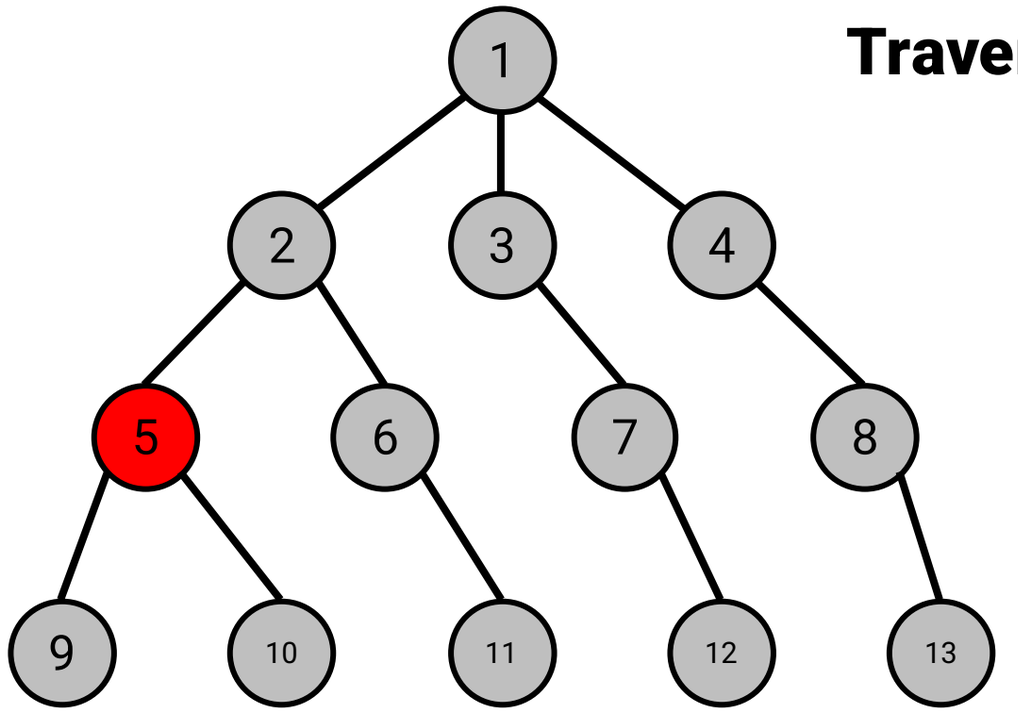
Breadth First Search



Traverse through Siblings First!

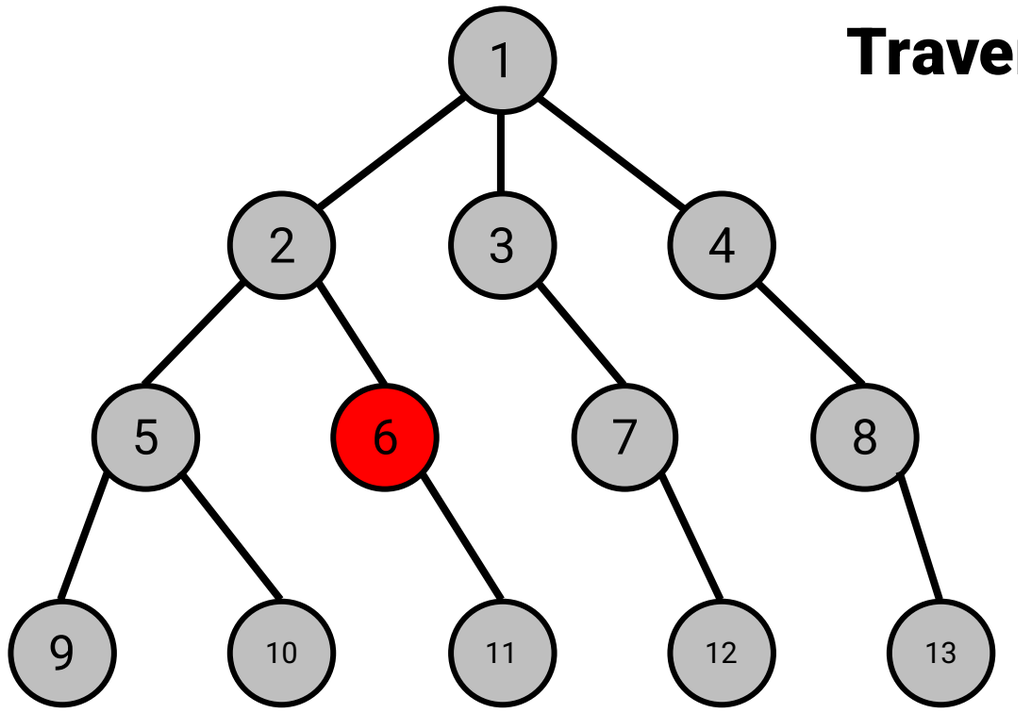
What's the next node we explore?

Breadth First Search



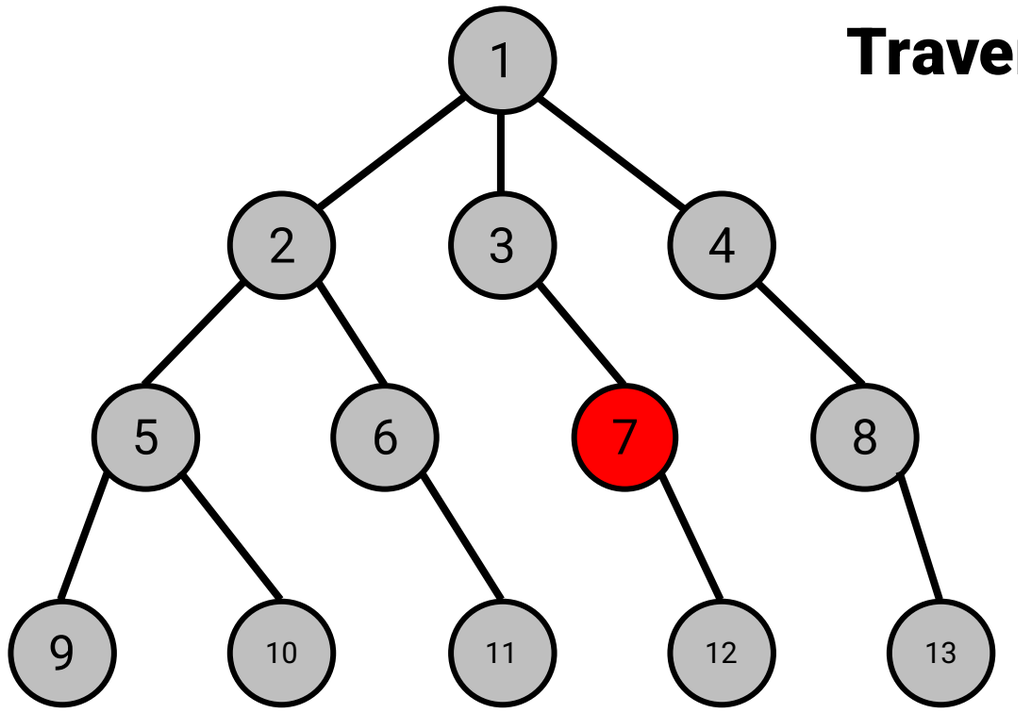
Traverse through Siblings First!

Breadth First Search



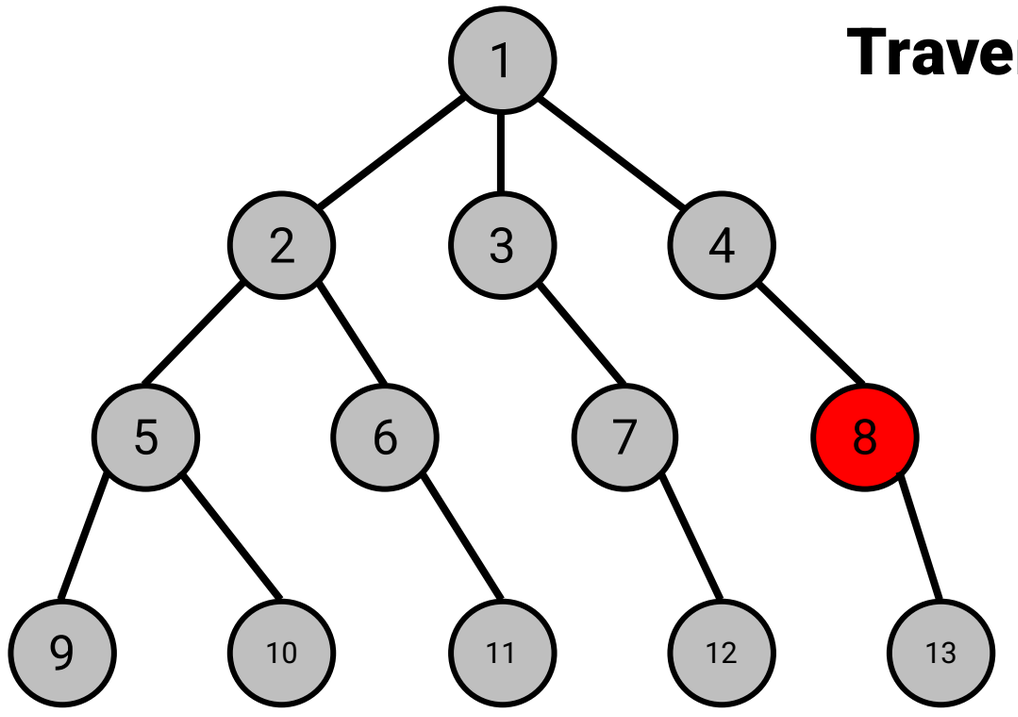
Traverse through Siblings First!

Breadth First Search



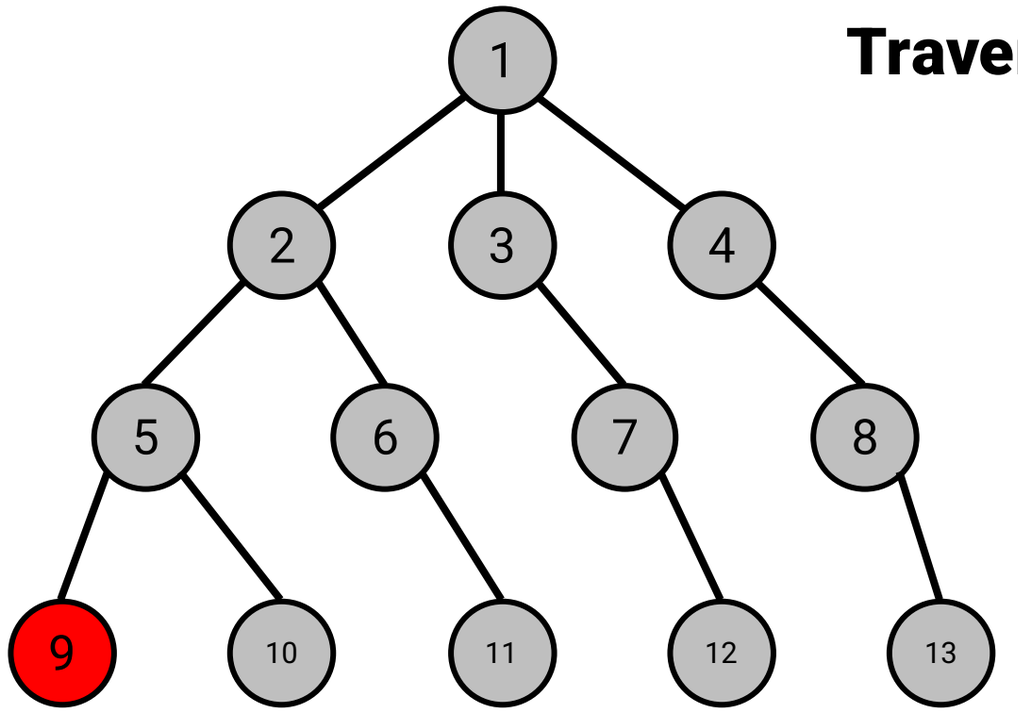
Traverse through Siblings First!

Breadth First Search



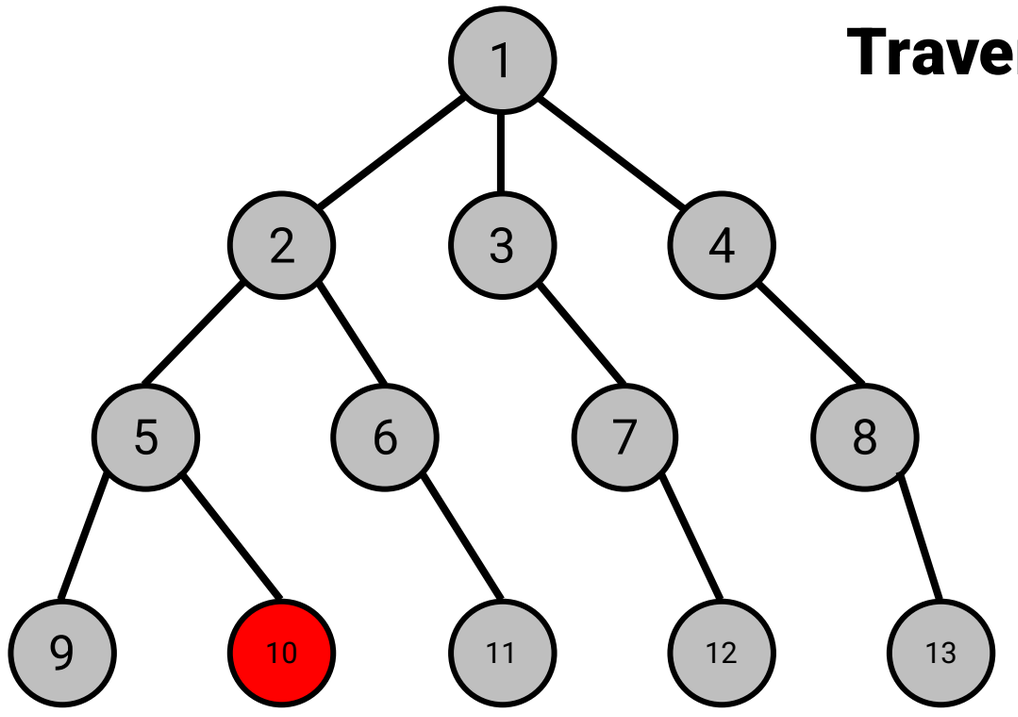
Traverse through Siblings First!

Breadth First Search



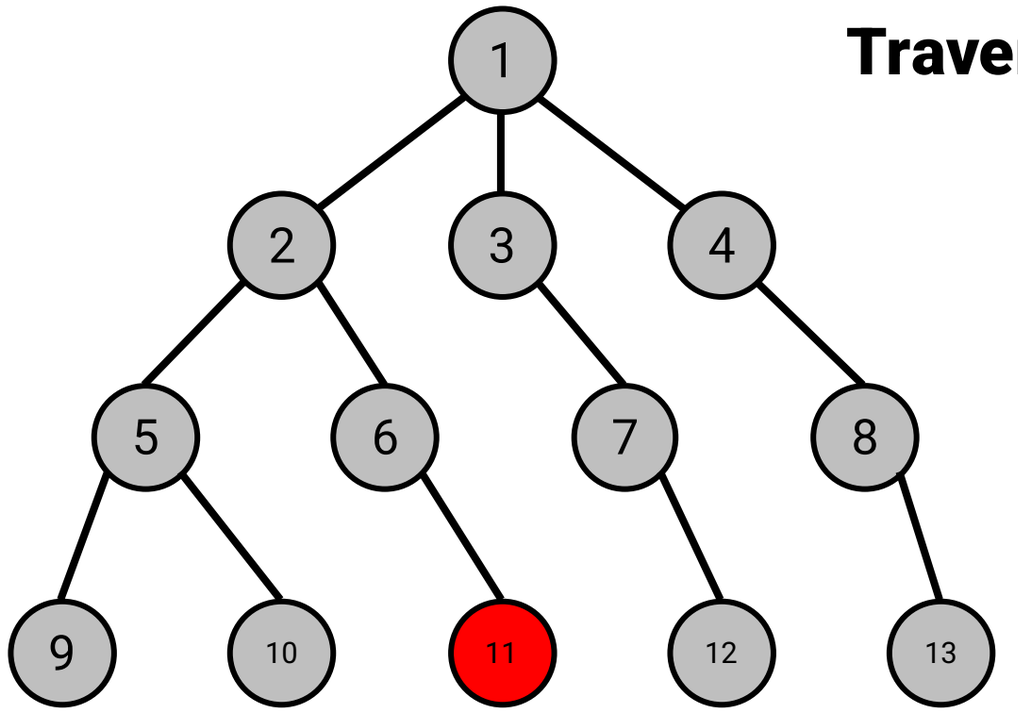
Traverse through Siblings First!

Breadth First Search



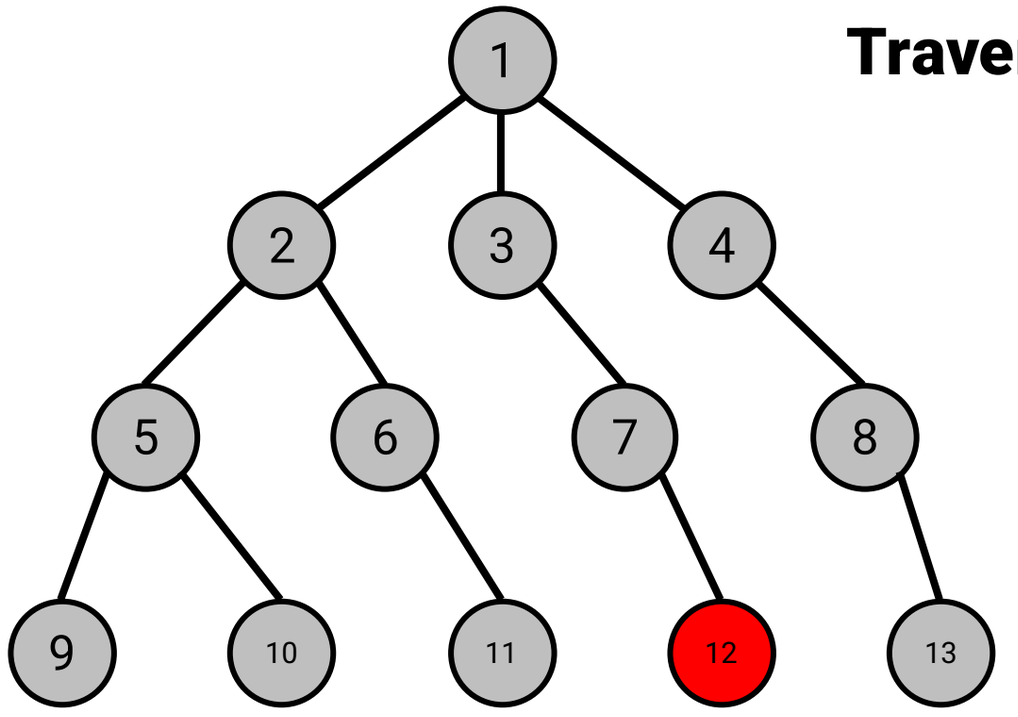
Traverse through Siblings First!

Breadth First Search



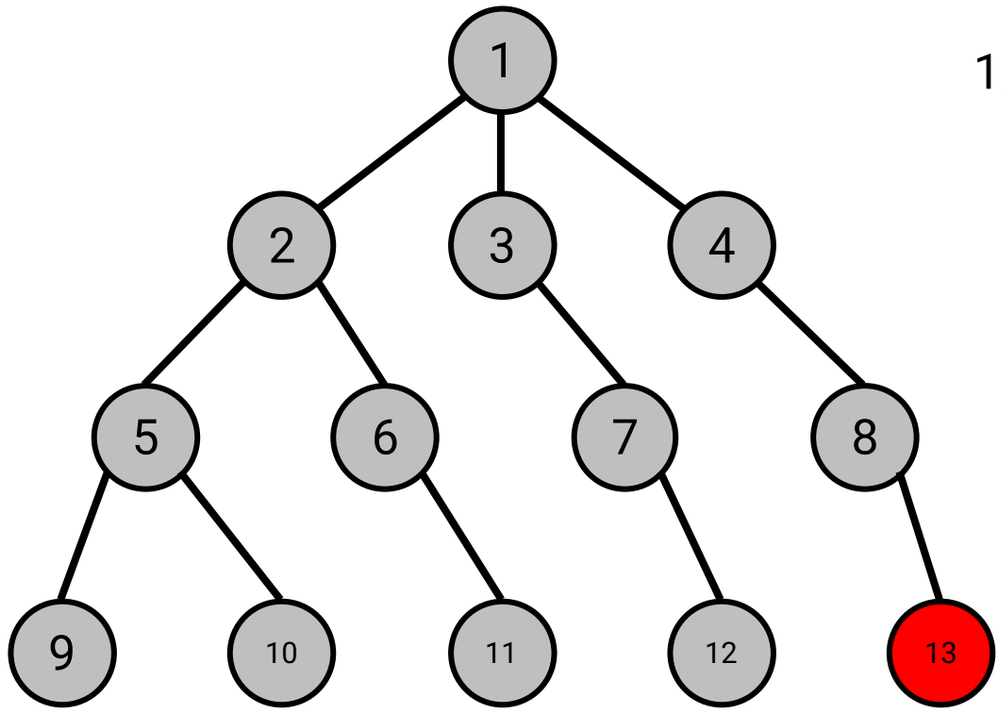
Traverse through Siblings First!

Breadth First Search



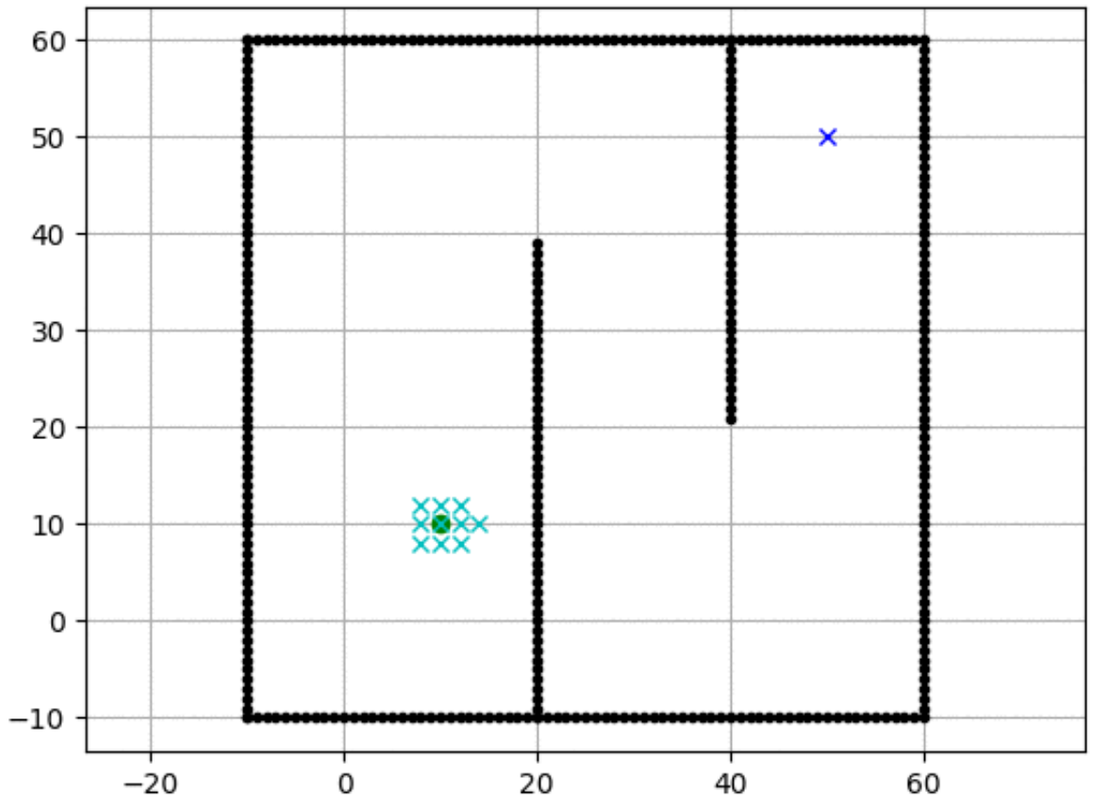
Traverse through Siblings First!

Breadth First Search

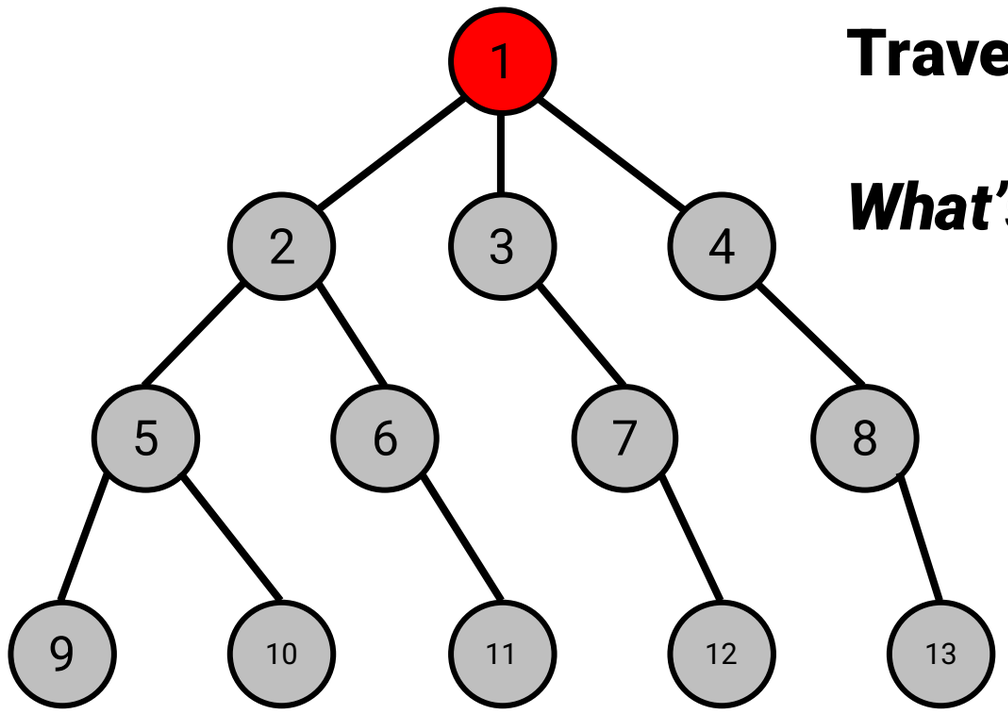


1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12 ,13

Breadth First Search



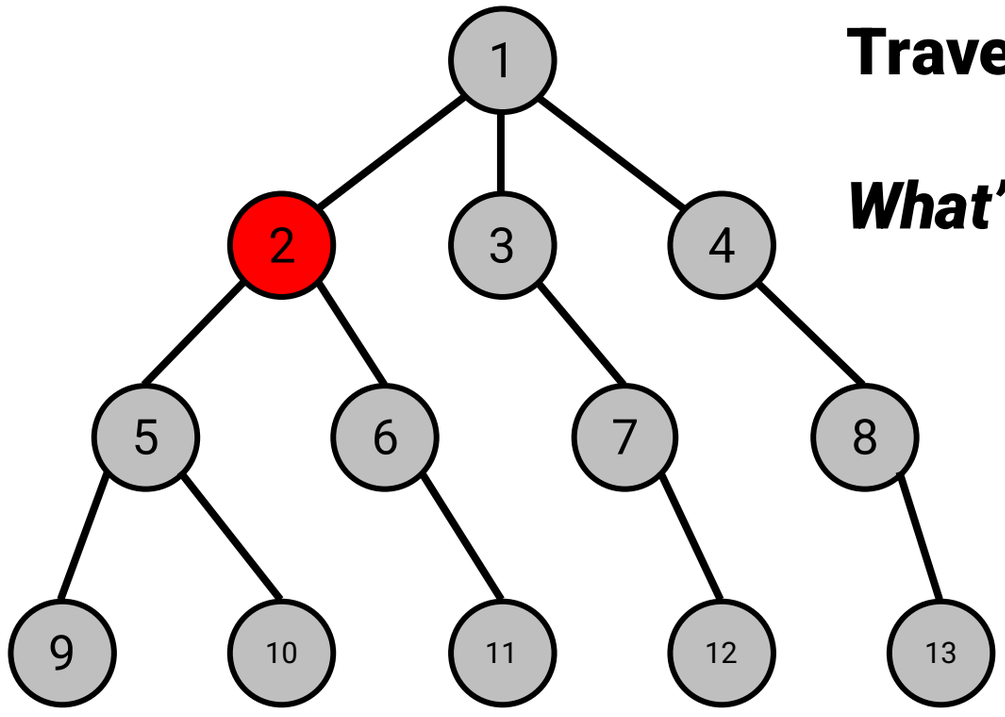
Depth First Search—Let’s try this on your own!



Traverse through Children First!

What’s the next node we explore?

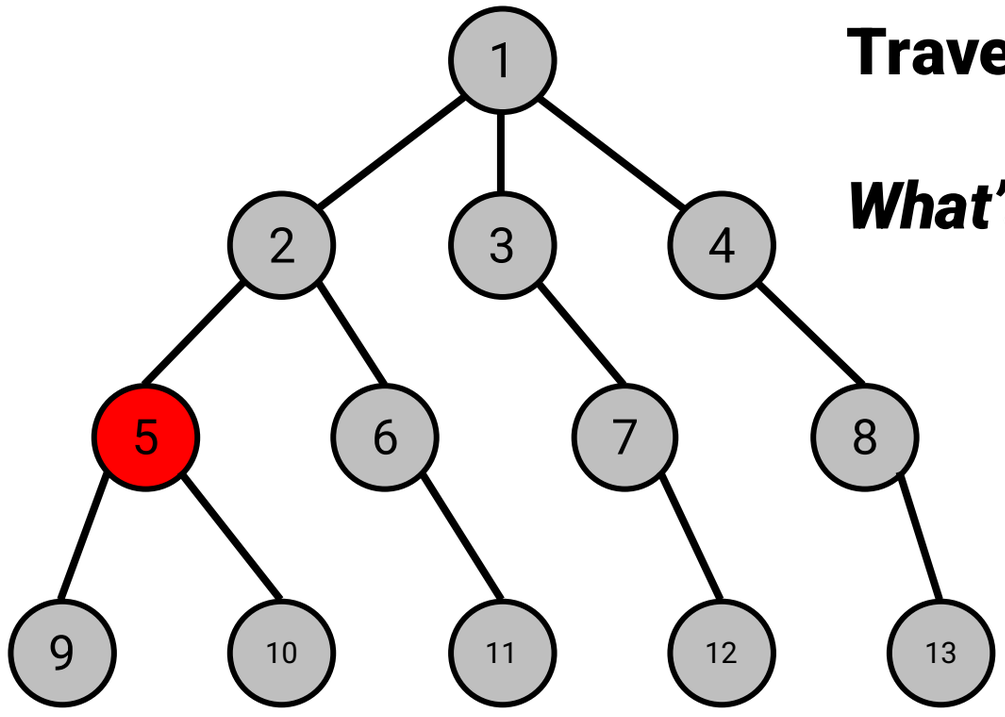
Depth First Search



Traverse through Children First!

What's the next node we explore?

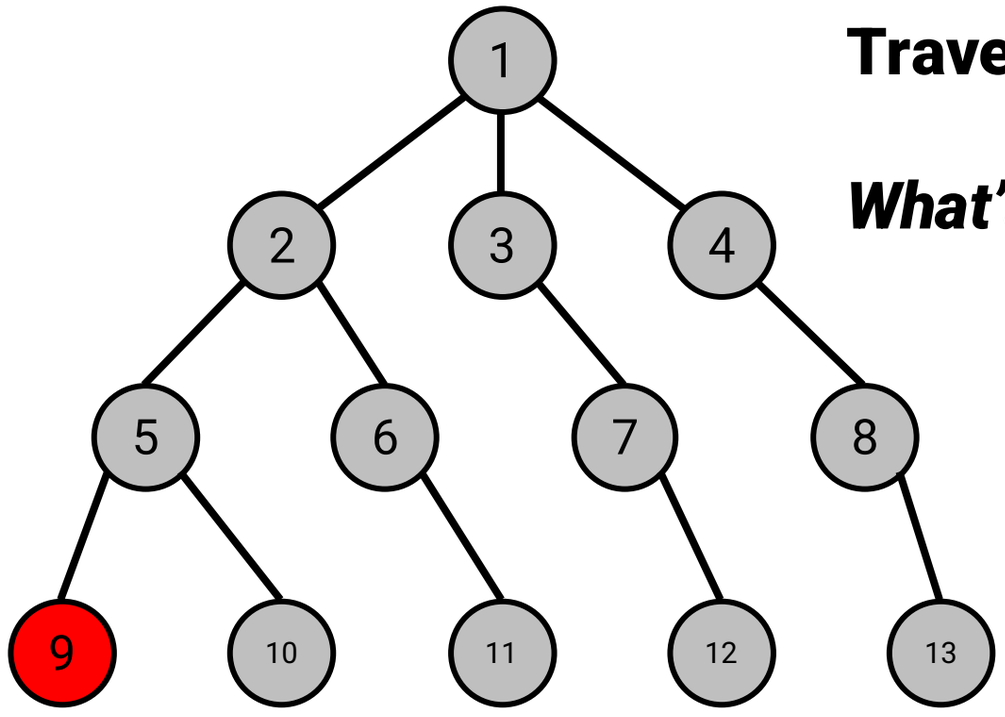
Depth First Search



Traverse through Children First!

What's the next node we explore?

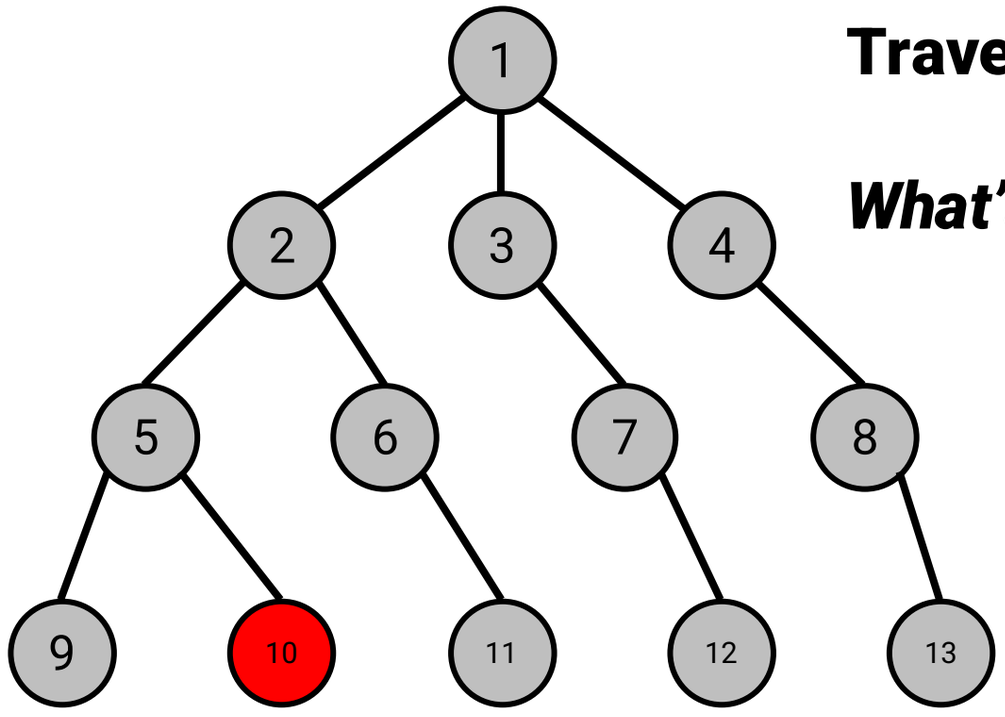
Depth First Search



Traverse through Children First!

What's the next node we explore?

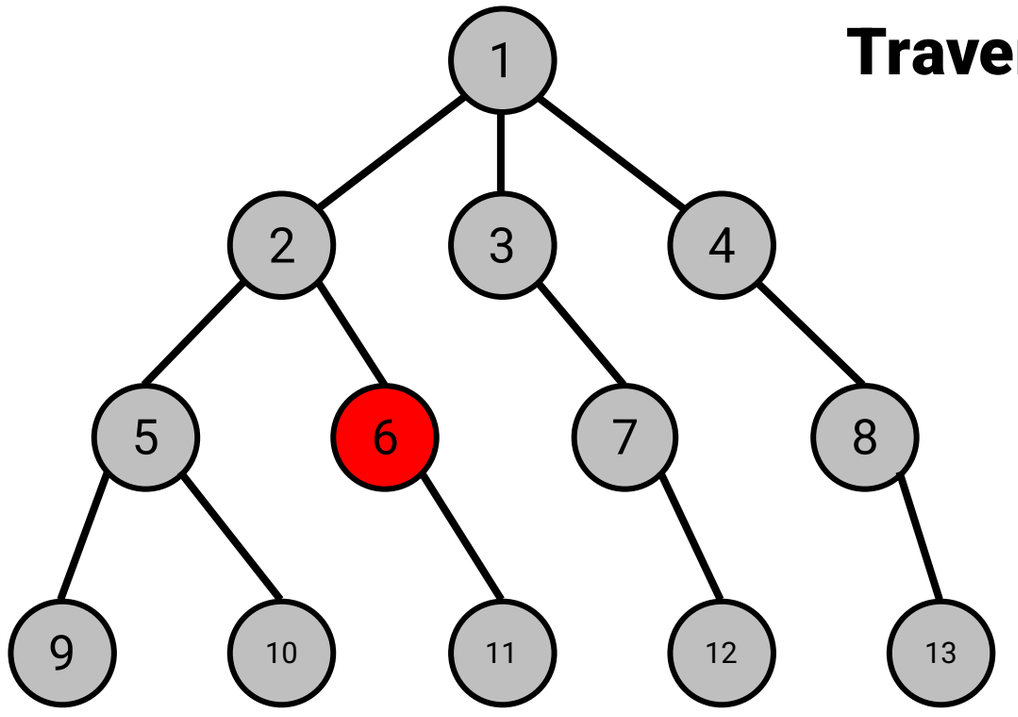
Depth First Search



Traverse through Children First!

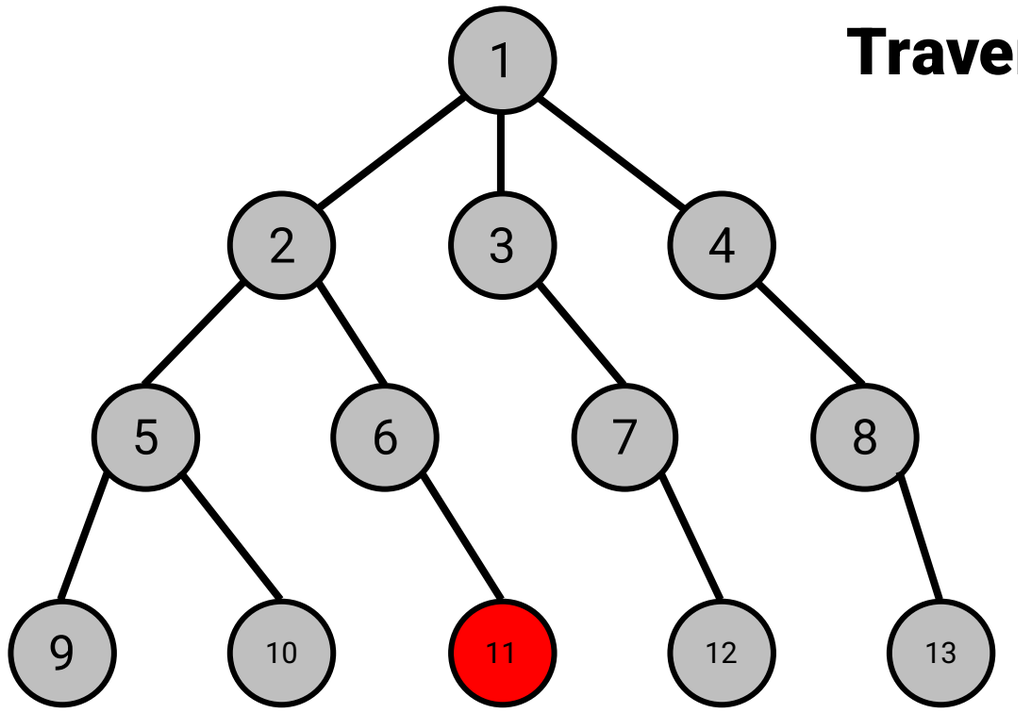
What's the next node we explore?

Depth First Search



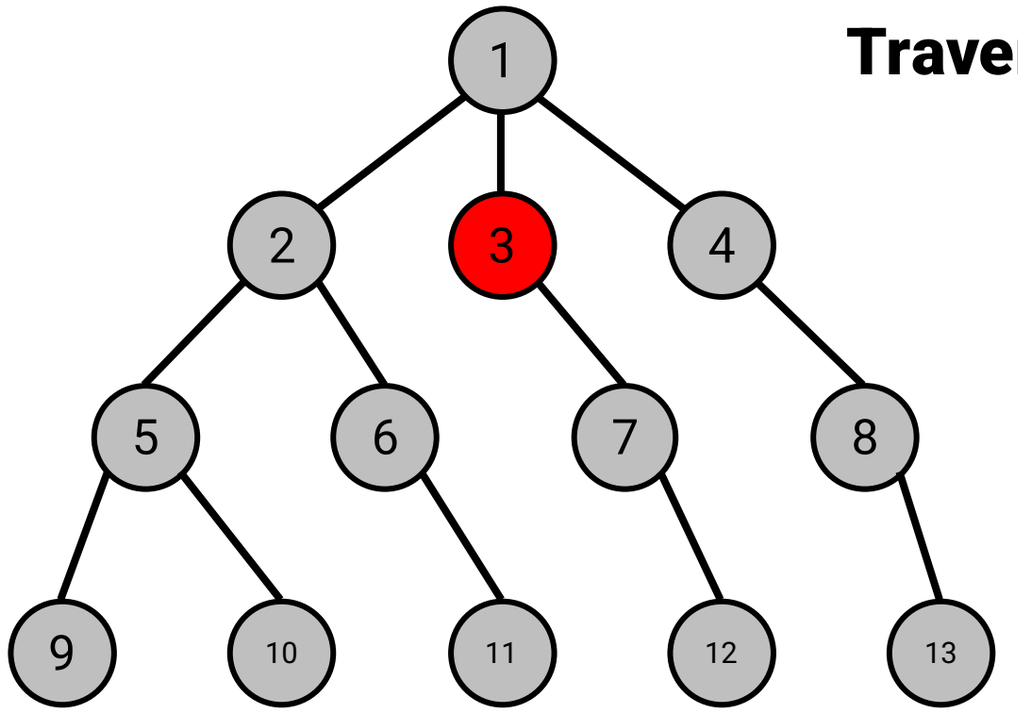
Traverse through Children First!

Depth First Search



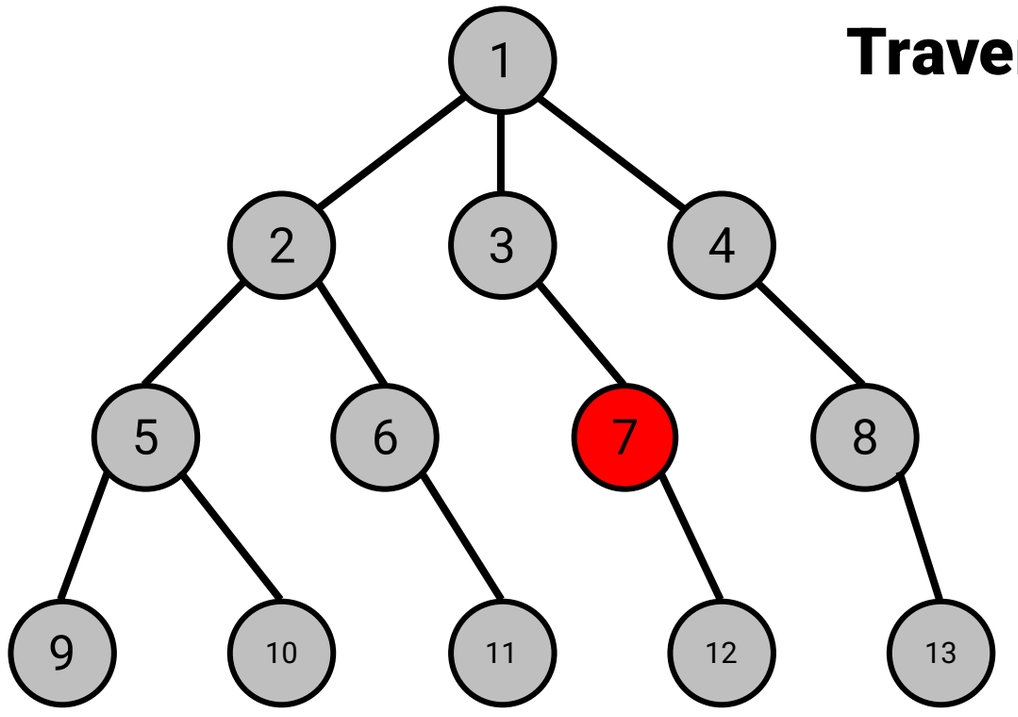
Traverse through Children First!

Depth First Search



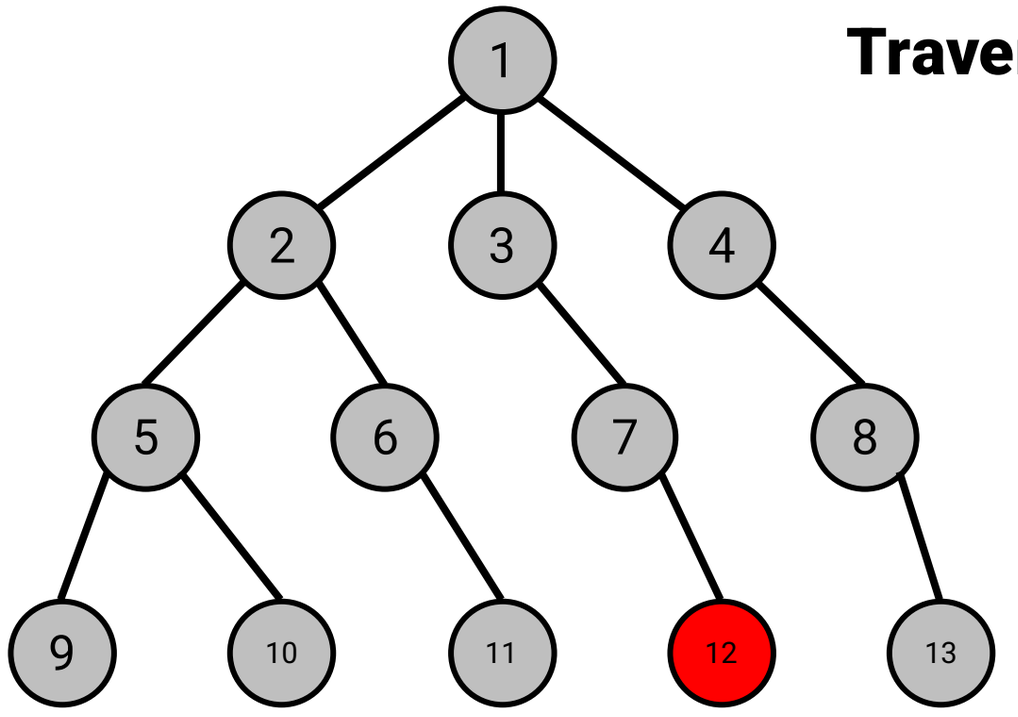
Traverse through Children First!

Depth First Search



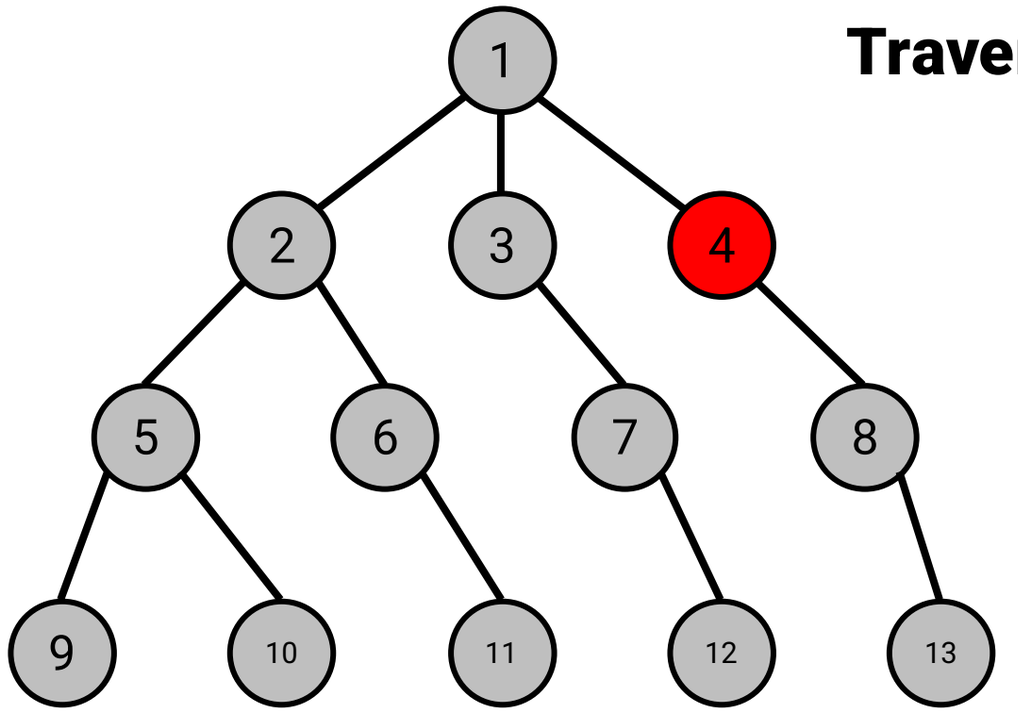
Traverse through Children First!

Depth First Search



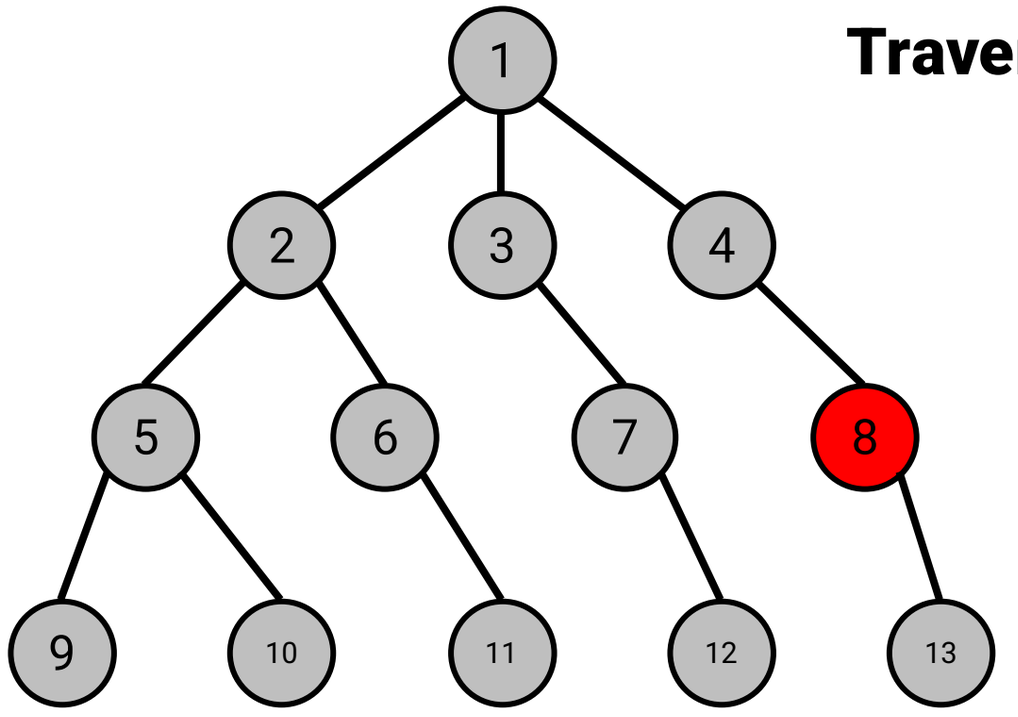
Traverse through Children First!

Depth First Search



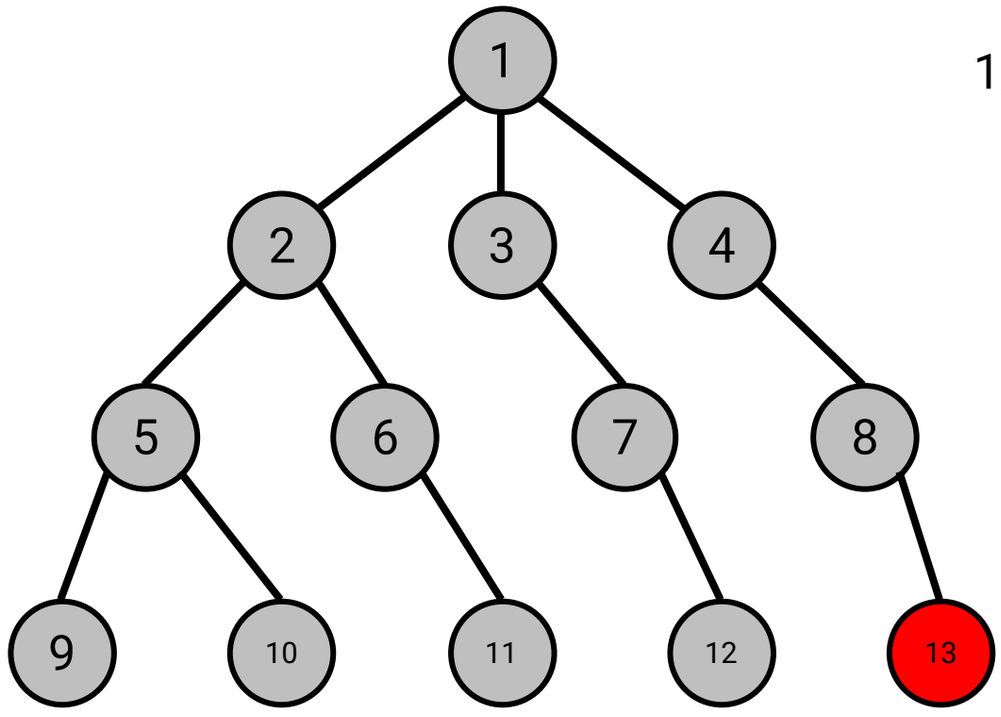
Traverse through Children First!

Depth First Search



Traverse through Children First!

Depth First Search



1, 2, 5, 9, 10, 6, 11, 3, 7, 12, 4, 8, 13

Depth First Search